



DCC

DEPARTAMENTO DE CIENCIA
DE LA COMPUTACIÓN

IIC2343

Arquitectura de Computadores

Clase 14 - Resumen del Curso

Profesor: Germán Leandro Contreras Sagredo

Propósito del material

- El objetivo de este material es que puedan realizar un estudio **rápido** de los contenidos más relevantes del curso.
- La idea es que con este puedan identificar **qué contenidos les falta profundizar**, de ser el caso, se espera que no solo se queden con lo que se encuentra aquí, sino que además acudan a las clases y apuntes correspondientes.
- También se espera que sirva como guía rápida mientras estén desarrollando ejercicios de los compilados.

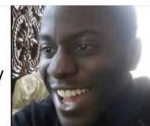


Representaciones numéricas

■ Contenido que encuentran en:

- **Clase 1 - Representaciones Numéricas I**
- **Clase 6 - Representaciones Numéricas II**
- **01 - Representaciones Numéricas Parte 1 - Números Enteros (Apuntes)**
- **02 - Representaciones Numéricas Parte 2 - Números Racionales (Apuntes)**

Your crush
knows binary



They think
you're a 10



They think
you're a 10



Representaciones numéricas - Números enteros

- En este curso, nos enfocamos en la **representación posicional binaria** por su utilidad en circuitos digitales y, posteriormente, componentes del computador básico.
- **Fórmula general de transformación de bases**

$$\sum_{k=0}^{n-1} s_k \times b^k$$

s = Símbolo (dígito).

n = Cantidad de dígitos en la secuencia.

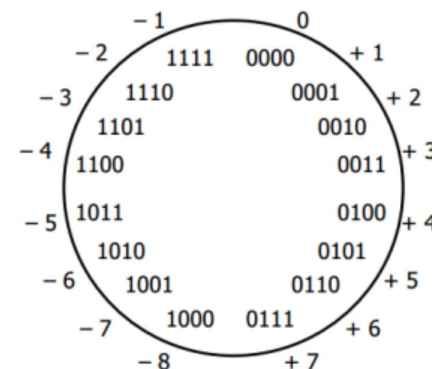
b = Base numérica (o número de dígitos).

k = Posición del dígito en la secuencia,
siendo 0 la posición del extremo derecho.

Representaciones numéricas - Números enteros

- **Complemento de 2:** Representación utilizada para números **enteros**. Se obtiene el complemento de 1 (complemento de cada dígito del número) y se suma una unidad al final. Esto asegura que $x + C_2(x) = 0$ (el bit de *carry* restante se descarta).
- **Contras del complemento de 2**
 - Representación **desbalanceada** (un número negativo adicional).
 - **Overflow:** Operación cuyo resultado no es representable con la cantidad de bits disponible resultan en un valor erróneo.

N	0101
$-N$	1011
$N + (-N)$	0000



Representaciones numéricas - Números enteros

■ Conversión entre base binaria y hexadecimal

- **Hexadecimal a binario:** Cada dígito hexadecimal se representa en su valor binario con cuatro dígitos. Se concatenan los resultados para el valor final.

$$0x9F2 = \left\{ \begin{array}{lcl} 0x9 & = & 1001b \\ 0xF & = & 1111b \\ 0x2 & = & 0010b \end{array} \right\} \rightarrow 0x9F2 = 100111110010b$$

- **Binario a hexadecimal:** Se agrupan cuatro dígitos binarios y se representan en el valor hexadecimal. Se concatenan los resultados para el valor final.

$$100111110010b = (1001)(1111)(0010) = (0x9)(0xF)(0x2) = 0x9F2$$

* Se puede hacer lo mismo entre la base binaria y la base octal, haciendo las conversiones con 3 dígitos en vez de 4.

Representaciones numéricas - Números reales

- Se pueden representar en base binaria, pero **números finitos en base decimal pueden ser infinitos en base binaria**. Esto hace que la cantidad de bits para representar estos números sea significativa para reducir el error.
- **Representaciones de números reales en base binaria**
 - **Punto fijo:** Dados N bits, estos se dividen de forma **fija** para representar signo, parte entera y parte decimal.
Ej: $N = 8$ bits = 1 bit signo (s) + 3 bits parte entera (t) + 4 bits fracción (f)
 $10,110b = \underset{s}{0} \underset{t}{010} \underset{f}{0110}$ $-1,0011b = \underset{s}{1} \underset{t}{001} \underset{f}{0011}$
 - **Punto flotante:** Dados N bits, los dividimos en un **significante/mantisa** y un **exponente**, ambos con signo.
Ej: $N = 8$ bits = 1 bit signo significativo (ss) + 3 bits significativo (s) + 1 bit signo exponente (se) + 3 bits exponente (e) $\rightarrow -0,00011b = -1,1 * 10^{-100} = \underset{ss}{1} \underset{s}{110} \underset{se}{1} \underset{e}{100}$

Representaciones numéricas - Números reales

- Usamos punto flotante por un **mayor rango** (en perjuicio de la precisión).
- Representación de punto flotante más utilizada: **Estándar IEEE754**
 - **Float (Single Precision Floating Point)**
 - 1 bit de signo.
 - 8 bits de exponente **desfasado** en 127 para obviar el bit de signo.
 - 23 bits de significante/mantisa **normalizado**: Tiene precisión de 24 bits ya que **asume que todo significante parte en 1**.
 - Exponente 00000000 reservado para representar el 0 (significante con solo bits de 0) y el exponente 11111111 para +-Infinity (significante con solo bits de 0) y NaN (significante con al menos un bit igual a 1).
 - $X = (-1)^{\text{signo}} * 1.\text{significante} * 10^{(\text{exponente} - 127)_b}$
Ej: $0.00101_b = (-1)^0 * 1.01 * 10^{01111100} =$
00111110001000000000000000000000 IEEE754

Representaciones numéricas - Números reales

- Representación de punto flotante más utilizada: **Estándar IEEE754**
 - **Double (*Double Precision Floating Point*)**
 - 1 bit de signo.
 - 11 bits de exponente **desfasado** en 1023 para obviar el bit de signo.
 - 52 bits de significante/mantisa **normalizado**: Tiene precisión de 53 bits ya que **asume que todo significativo parte en 1**.
 - Presenta un error de precisión mucho menor a float (error de float igual a 10^{-23} ; error de double igual a 10^{-52}), pero requiere del doble de bits para ser representado.

Representaciones numéricas - Números reales

■ Operaciones sobre números de punto flotante

○ Suma

1. Igualar exponentes. Se toma el número de menor exponente y este se incrementa hasta igualar al mayor. Al hacer esto, se mueve el punto decimal del significante.
2. Sumar los significantes.
3. Normalizar el resultado.
4. Pasar a formato float.

$$\begin{aligned} &1,11 * 10^{11} + 1,11 * 10^1 \\ &= 1,11 * 10^{11} + \mathbf{0,0111} * 10^{11} \\ &= (\mathbf{1,11} + \mathbf{0,0111}) * 10^{11} \\ &= 10,0011 * 10^{11} \\ &= \mathbf{1,00011} * 10^{100} \end{aligned}$$

○ Multiplicación

1. Sumar exponentes **sin considerar el desfase dos veces**.
2. Multiplicar significantes. Se pueden multiplicar ignorando el punto decimal; luego, se vuelve a posicionar según la suma de las posiciones decimales de ambos operadores.
3. Normalizar el resultado.
4. Pasar a formato float.

$$\begin{aligned} &1,11 * 10^{11} + 1,1 * 10^{11} \\ &= (1,11 * 1,1) * \mathbf{10^{110}} \\ &= \mathbf{10,101} * 10^{110} \\ &= \mathbf{1,0101} * 10^{111} \end{aligned}$$

Representaciones numéricas - Números racionales

- Ejemplo de pérdida de precisión en floats: $2^{30} + 5$

$$2^{30} \rightarrow (1)^0 * 1.0 * 10^{(157 - 127)} = 1 * 1.0 * 10^{157b} = 1^0 * 1.0 * 10^{10011101}$$

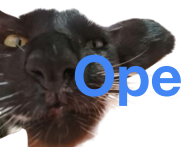
$$(2^{30})_{\text{IEEE754}} = \text{01001110100000000000000000000000}_{\text{IEEE754}}$$

$$\begin{aligned} 5 &\rightarrow (1)^0 * 1.01 * 10^{(129 - 127)} = 1 * 1.01 * 10^{129b} = 1^0 * 1.01 * 10^{10000001} \\ (5)_{\text{IEEE754}} &= \text{01000000101000000000000000000000}_{\text{IEEE754}} \end{aligned}$$

- Para sumarlos, igualamos la potencia del número menor a la del mayor:

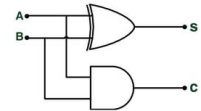
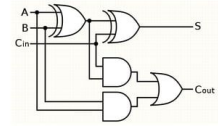
[illegible]

* Estos bits se descartan porque la mantisa acepta hasta 23 bits.



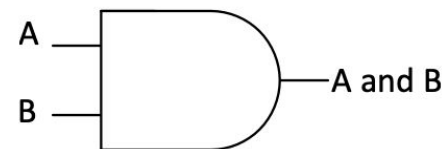
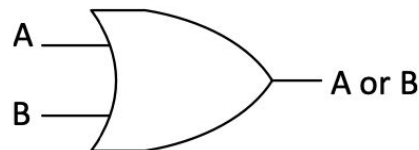
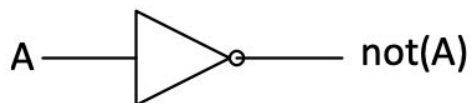
Operaciones aritméticas y lógicas

- Contenido que encuentran en:
 - **Clase 2 - Operaciones Aritméticas y Lógicas**
 - **03 - Operaciones Aritméticas y Lógicas**



Operaciones aritméticas y lógicas - Compuertas lógicas

- **Compuertas lógicas:** Base de todos los componentes del computador básico: NOT, OR, AND.



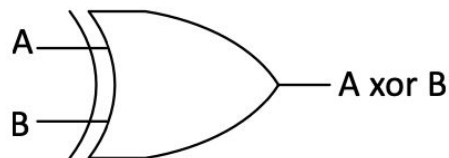
A	not(A)
1	0
0	1

A	B	A or B
1	1	1
1	0	1
0	1	1
0	0	0

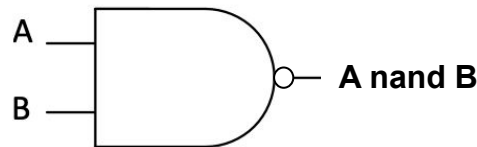
A	B	A and B
1	1	1
1	0	0
0	1	0
0	0	0

Operaciones aritméticas y lógicas - Puertas lógicas

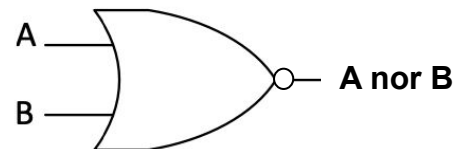
- **Compuertas lógicas:** Base de todos los componentes del computador básico: XOR, NAND, NOR, XNOR.



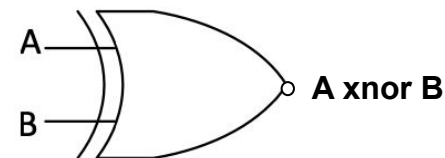
A	B	$A \text{ xor } B$
1	1	0
1	0	1
0	1	1
0	0	0



A	B	$A \text{ nand } B$
1	1	0
1	0	1
0	1	1
0	0	1



A	B	$A \text{ nor } B$
1	1	0
1	0	0
0	1	0
0	0	1



A	B	$A \text{ xnor } B$
1	1	1
1	0	0
0	1	0
0	0	1

Operaciones aritméticas y lógicas - *Minterms* y *maxterms*

- Técnicas utilizadas para obtener la expresión lógica equivalente a una tabla de verdad.
- La expresión resultante se puede traducir directamente a un circuito eléctrico al utilizar solo conectivos lógicos AND, OR, NOT.
- Los *minterms* se basan en las condiciones (señales de entrada) que derivan en un valor de verdad igual a 1; los *maxterms* en las que derivan en un valor de verdad igual a 0. Conviene usar *minterms* cuando menos de la mitad de las filas de la tabla de verdad tienen *output* igual a 1; en otro caso, conviene usar *maxterms*. Si se tiene la misma cantidad de filas con *output* igual a 0 y 1, entonces se puede usar cualquiera de los dos.

Operaciones aritméticas y lógicas - *Minterms y maxterms*

- **Minterms:** Solo se consideran las filas cuyo valor de verdad sea 1. Las condiciones de cada fila de la tabla se conectan con conectivos AND; las condiciones falsas (iguales a 0) se niegan. La expresión de cada fila se conecta con conectivos OR.

$(\text{NOT}(A) \text{ AND } B \text{ AND NOT}(C)) \text{ OR } (A \text{ AND NOT}(B) \text{ AND } C) \text{ OR } (A \text{ AND } B \text{ AND } C)$

$\equiv$

$$(\bar{A} \cdot B \cdot \bar{C}) + (A \cdot \bar{B} \cdot C) + (A \cdot B \cdot C)$$

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Operaciones aritméticas y lógicas - *Minterms y maxterms*

- **Maxterms:** Solo se consideran las filas cuyo valor de verdad sea 0. Las condiciones de una fila de la tabla se conectan con conectivos OR; las condiciones verdaderas (iguales a 1) se niegan. La expresión de cada fila se conecta con conectivos AND.

$$(A \text{ OR } B \text{ OR } C) \text{ AND } (A \text{ OR } B \text{ OR NOT}(C)) \text{ AND } (A \text{ OR NOT}(B) \text{ OR NOT}(C)) \\ \text{AND } (\text{NOT}(A) \text{ OR } B \text{ OR } C) \text{ AND } (\text{NOT}(A) \text{ OR NOT}(B) \text{ OR } C)$$

≡

$$(A+B+C) \cdot (A+B+\bar{C}) \cdot (A+\bar{B}+\bar{C}) \cdot (\bar{A}+B+C) \cdot (\bar{A}+\bar{B}+C)$$

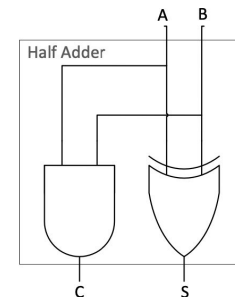
A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

* También se puede construir usando el mismo formato de *minterms*, pero aplicando una negación al final y aplicando Ley De Morgan.

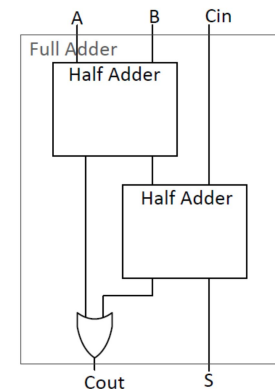
Operaciones aritméticas y lógicas - Componentes aritméticas

- **Half-Adder:** “Medio sumador”, suma dos bits sin considerar bit de *carry*.

A	B	S	C
1	1	0	1
1	0	1	0
0	1	1	0
0	0	0	0



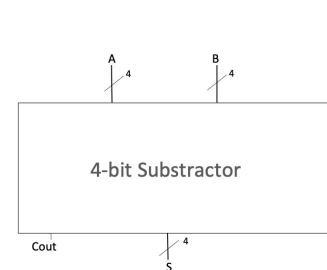
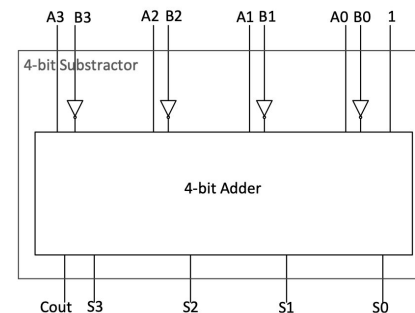
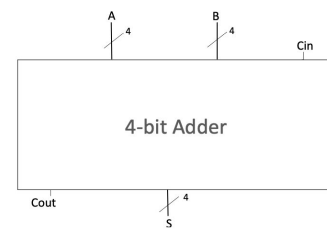
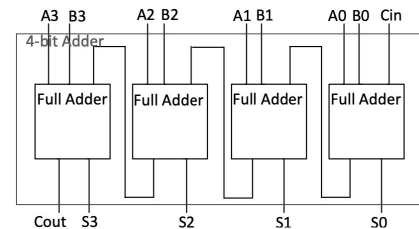
- **Full-Adder:** Sumador completo, suma dos bits y considera bit de *carry* (señal C_{in}).



* El *carry* de salida puede ser generado con la suma de A y B o con la suma entre dicho resultado y el *carry* de entrada, razón por la que C_{out} es la compuerta OR de estos dos resultados

Operaciones aritméticas y lógicas - Componentes aritméticos

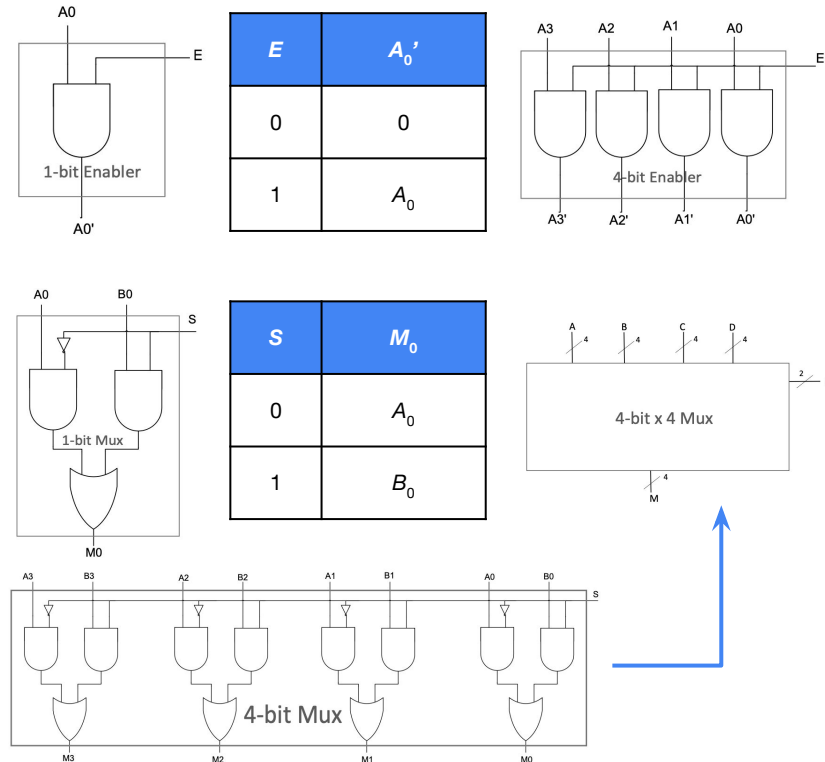
- **Sumador de 4 bits:** *Full-Adders* conectados a través de la señal C_{in} . Se extiende a más bits de la misma forma.
- **Restador de 4 bits:** Sumador de 4 bits pero con entrada B transformada a su complemento de dos.



* Notación de buses utilizada para facilitar la abstracción de la cantidad de señales.

Operaciones aritméticas y lógicas - Componentes aritméticos

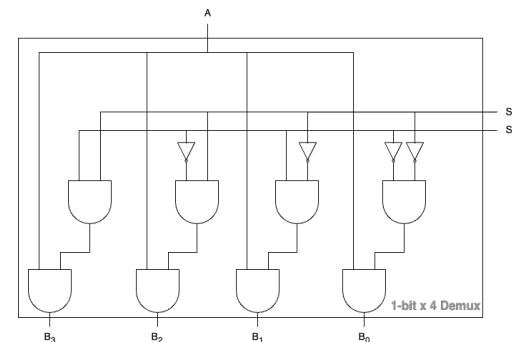
- **Enabler:** Componente que habilita o no el paso de una señal.
- **Multiplexor/Mux:** Componente que selecciona como salida una de un conjunto de señales de entrada.



Operaciones aritméticas y lógicas - Componentes aritméticas

■ De-Multiplexor/Demux:

Componente que transmite una señal de entrada a una de múltiples salidas. En el resto de salidas no seleccionadas se transmite un 0.

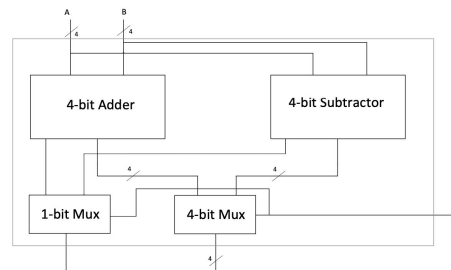


S_1	S_0	Output
0	0	$B_0 = A$
1	0	$B_1 = A$
0	1	$B_2 = A$
1	1	$B_3 = A$

Operaciones aritméticas y lógicas - Componentes aritméticas

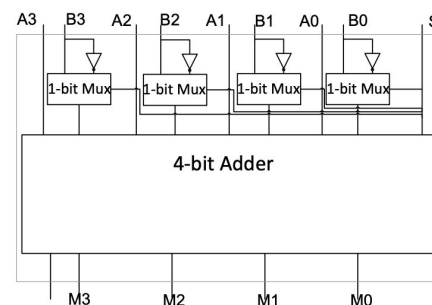
■ Sumador-Restador

- No optimizado.



S	M	C _{out}
0	$A+B$	$A+B C_{out}$
1	$A-B$	$A-B C_{out}$

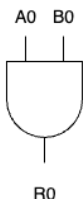
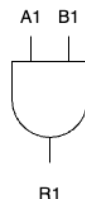
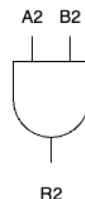
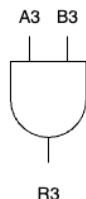
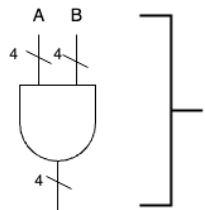
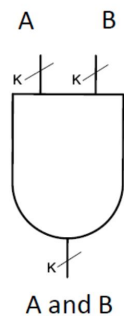
- Optimizado.



S	M	C _{out}
0	$A+B$	$A+B C_{out}$
1	$A-B$	$A-B C_{out}$

Operaciones aritméticas y lógicas - Componentes aritméticas

- **Operadores *bitwise*:** Componentes que extienden las operaciones de compuertas lógicas a señales de más de 1 bit.



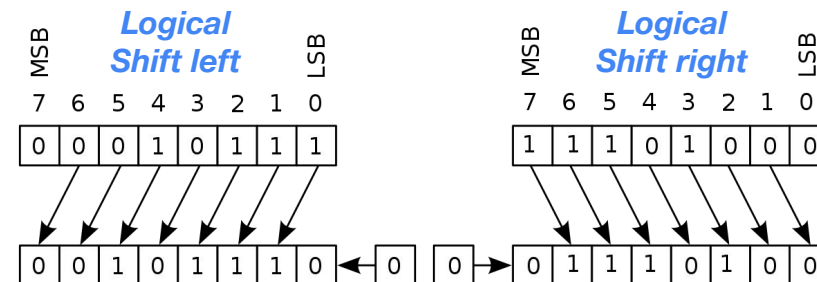
$$R = (A3 \text{ AND } B3)(A2 \text{ AND } B2)(A1 \text{ AND } B1)(A0 \text{ AND } B0) = R3R2R1R0$$

* Ejemplo de construcción de componente *bitwise* AND.

Operaciones aritméticas y lógicas - Componentes aritméticas

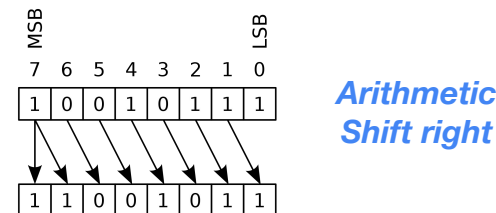
- **Logical shifting components:** Componentes que desplazan los bits de un número a la derecha o a la izquierda, equivalente a dividir o multiplicar por 2 de forma respectiva.

- `shift_left(0100b)` = 1000b
- `shift_left(1001b)` = 0010b
- `shift_right(0100b)` = 0010b
- `shift_right(1001b)` = 0100b



- **Arithmetic shift right:** Componente que realiza *shift right* manteniendo el signo (bit más significativo).

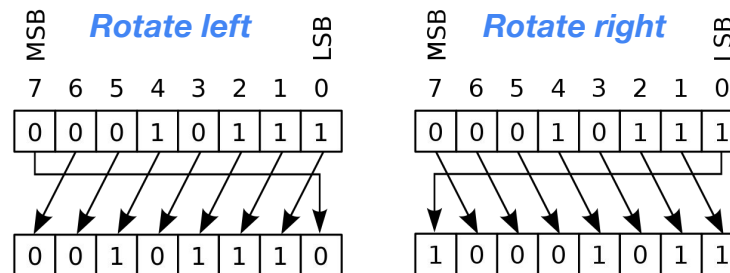
- `shift_arithmetic_right(1100b)` = 1110b
- `shift_arithmetic_right(1001b)` = 1100b



Operaciones aritméticas y lógicas - Componentes aritméticas

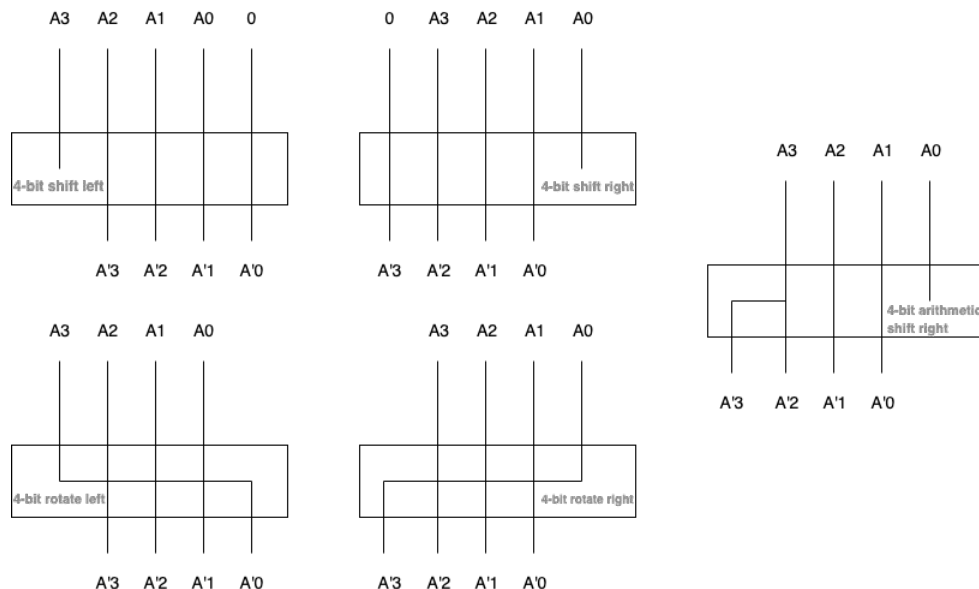
- **Rotating components:** Componentes que desplazan los bits de un número a la derecha o a la izquierda, pero que rotan el bit “descartado” al bit más o menos significativo respectivamente.

- `rotate_left(1000b)` = 0001b
- `rotate_left(0101b)` = 1010b
- `rotate_right(0100b)` = 0010b
- `rotate_right(0011b)` = 1001b



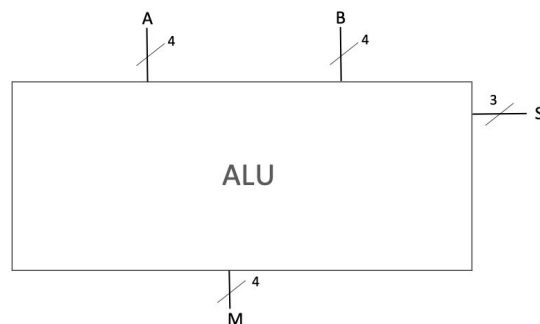
Operaciones aritméticas y lógicas - Componentes aritméticas

■ Diagramas de componentes de *shift* y *rotate*



Operaciones aritméticas y lógicas - Componentes aritméticas

- **Arithmetic Logic Unit (ALU):** Realiza las operaciones aritmético-lógicas con los componentes antes vistos y selecciona la operación mediante multiplexores (que reciben el resultado de todas las operaciones). Será la **unidad de ejecución del computador básico**.



S2	S1	S0	M
0	0	0	Suma
0	0	1	Resta
0	1	0	And
0	1	1	Or
1	0	0	Not
1	0	1	Xor
1	1	0	Shift left
1	1	1	Shift right

* Solo realiza *shifts* lógicos, no aritméticos.

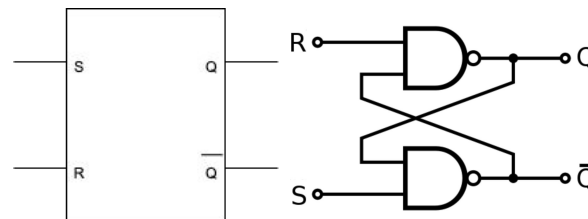
Almacenamiento de datos

- Contenido que encuentran en:
 - **Clase 3 - Almacenamiento de Datos**
 - **04 - Almacenamiento de Datos (Apuntes)**



Almacenamiento de datos - Latches

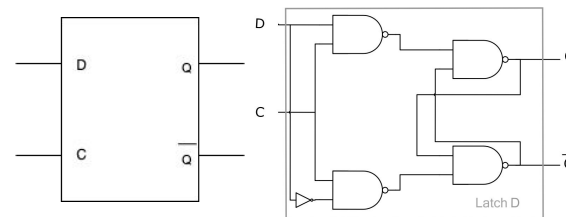
- **Latch RS:** Componente que puede almacenar un estado o cambiarlo mediante las señales *R*(eset) y *S*(et) a través de un **circuito secuencial**.



S	R	$Q(t+1)$
0	0	-
0	1	0
1	0	1
1	1	$Q(t)$

* Notar el uso de compuertas NAND.

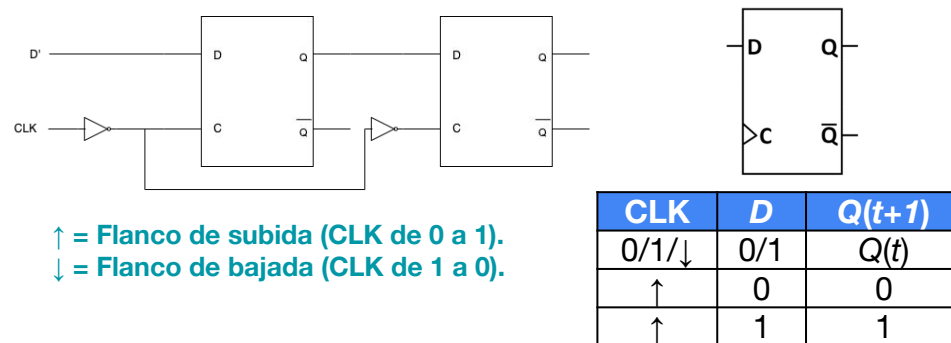
- **Latch D:** Componente que se construye sobre un Latch RS para permitir el almacenamiento de una señal *D* a partir de una señal de control *C*.



C	D	$Q(t+1)$
0	0	$Q(t)$
0	1	$Q(t)$
1	0	0
1	1	1

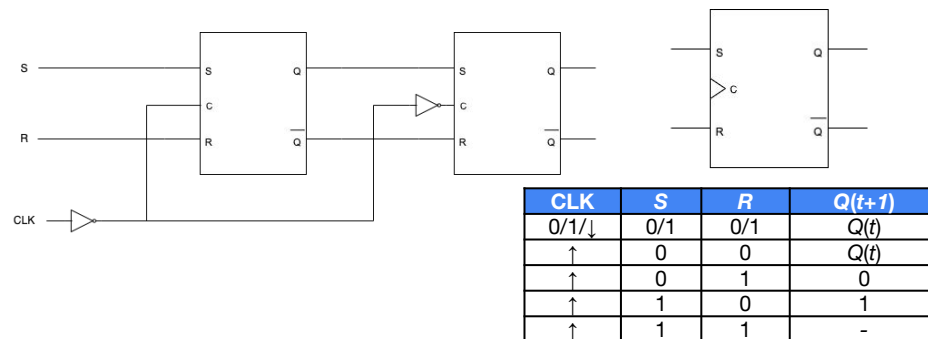
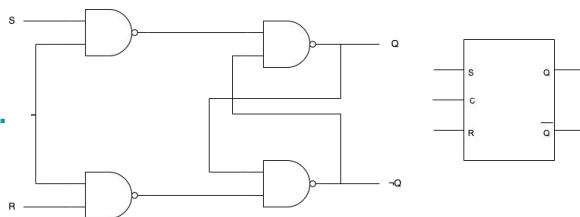
Almacenamiento de datos - Flip-flops

- **Flip-flop D:** Componente que permite guardar el estado anterior de una señal **en un instante dado**.



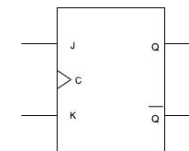
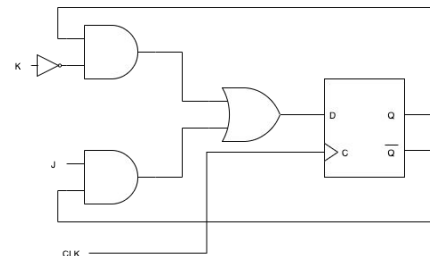
- **Flip-flop RS:** Flip-flop construido con latches RS que incluyen señal de control C.

* Latch RS con señal de control C.



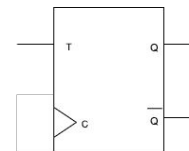
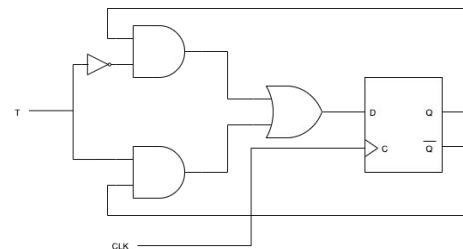
Almacenamiento de datos - Flip-flops

- **Flip-flop JK:** Similar al flip-flop RS, pero sin estado inválido.



CLK	J	K	$Q(t+1)$
0/1/↓	0/1	0/1	$Q(t)$
↑	0	0	$Q(t)$
↑	0	1	0
↑	1	0	1
↑	1	1	$\neg Q(t)$

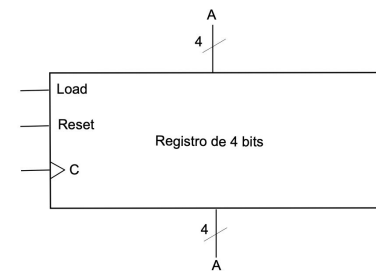
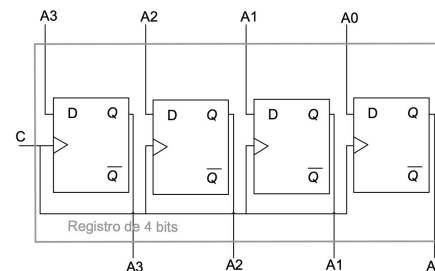
- **Flip-flop T:** Flip-flop que a partir de una señal T (ogg/e) invierte el estado.



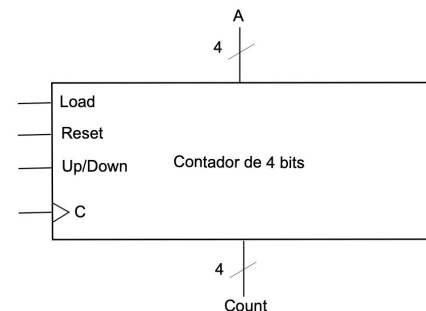
CLK	T	$Q(t+1)$
0/1/↓	0/1	$Q(t)$
↑	0	$Q(t)$
↑	1	$\neg Q(t)$

Almacenamiento de datos - Registros y contadores

- **Registro:** Conjunto de flip-flops que almacenan cada bit de un valor numérico. Posee señales *Reset* para dejar el valor en 0 y *Load* para cargar una señal de entrada.



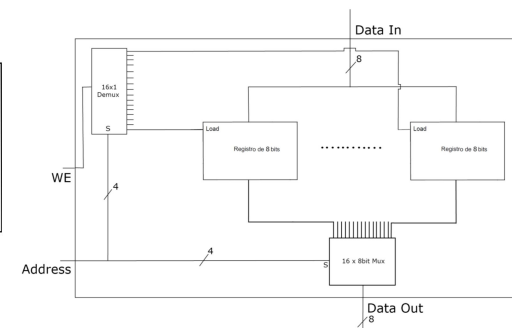
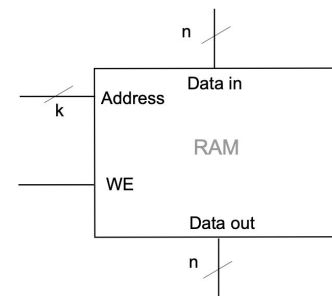
- **Contador:** Registro con señales de incremento/decremento para aumentar o disminuir en una unidad el valor almacenado.



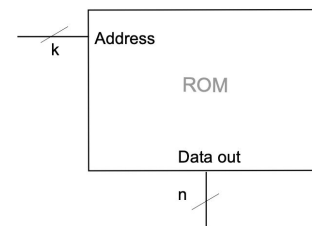
* De ahora en adelante, todo componente que en su interior posea flip-flops debe estar conectado al *clock* del sistema para asegurar sincronización.

Almacenamiento de datos - Memorias

- **RAM:** Extensión de los registros para leer o escribir **palabras de memoria** (generalmente de 8 bits - 1 byte). Si se poseen k bits de direccionamiento, la memoria posee 2^k palabras de memorias (2^k bytes).
- **ROM:** Memoria de “solo lectura” (dependiendo del caso, pero pueden asumirla así).



* El uso del Demux es fundamental para el funcionamiento de la memoria RAM y la propagación de la señal WE (*Write Enable*) solo al registro que corresponda.



Almacenamiento de datos - Tipos de dato

- **Variables:** Valor que puede cambiar durante la ejecución de un programa y cuyo tipo de dato incide en la forma en que se almacena.

Tipo de dato	Codificación	Interpretación	#Bits de representación
char	base 2 sin signo	carácter o entero positivo	8
signed char	base 2 con signo en complemento de 2	entero positivo o negativo	8
short	base 2 con signo en complemento de 2	entero positivo o negativo	16
unsigned short	base 2 sin signo	entero positivo	16
int	base 2 con signo en complemento de 2	entero positivo o negativo	32
unsigned int	base 2 sin signo	entero positivo	32
long	base 2 con signo en complemento de 2	entero positivo o negativo	64
unsigned long	base 2 sin signo	entero positivo	64
long long	base 2 con signo en complemento de 2	entero positivo o negativo	128
unsigned long long	base 2 sin signo	entero positivo	128
float	punto flotante de precisión simple	Racionales y casos especiales	32
double	punto flotante de precisión doble	Racionales y casos especiales	64
long double	punto flotante de precisión cuádruple	Racionales y casos especiales	128

Almacenamiento de datos - *Endianness*

- ***Endianness***: Orden en el que se almacenan la secuencia de un dato.
 - ***Big Endian***: La palabra más significativa de la secuencia se almacena en la dirección **menor**.
 - ***Little Endian***: La palabra menos significativa de la secuencia se almacena en la dirección **menor**.

* Ejemplos de *endianness* para el almacenamiento del dato 0000111101010101, que se separa en los bytes 00001111 y 01010101

Dirección de memoria (hexa)	Palabra almacenada (<i>big endian</i>)
0x00	00001111
0x01	01010101
0x02	-

Dirección de memoria (hexa)	Palabra almacenada (<i>little endian</i>)
0x00	01010101
0x01	00001111
0x02	-

Almacenamiento de datos - Arreglos y matrices

- **Arreglo:** Tipo de dato que además posee un **largo** y una **dirección de memoria de inicio**.
- **Matriz:** Arreglo de arreglos. Se puede almacenar según la convención de **filas** o **columnas**.

```
let variables: number[] = [1, 3, 5, 7];
```

Dirección de memoria (hexa)	Palabra almacenada
0x00	00000001
0x01	00000011
0x02	00000101
0x03	00000111
0x04	-

```
let variablesMatrix: number[][] = [
  [1, 3, 5],
  [2, 4, 6]
];
```

Dirección de memoria (hexa)	Palabra almacenada	Dirección de memoria (hexa)	Palabra almacenada
0x00	00000001	0x00	00000001
0x01	00000011	0x01	00000010
0x02	00000101	0x02	00000011
0x03	00000010	0x03	00000100
0x04	00000100	0x04	00000101
0x05	00000110	0x05	00000110

Direccionamiento en convención por filas: $dir(matriz[i,j]) = dir(matriz) + i * sizeof(matriz[i,j]) * \#columnas + j * sizeof(matriz[i,j])$

Direccionamiento en convención por columnas: $dir(matriz[i,j]) = dir(matriz) + j * sizeof(matriz[i,j]) * \#filas + i * sizeof(matriz[i,j])$

Computador básico

■ Contenido que encuentran en:

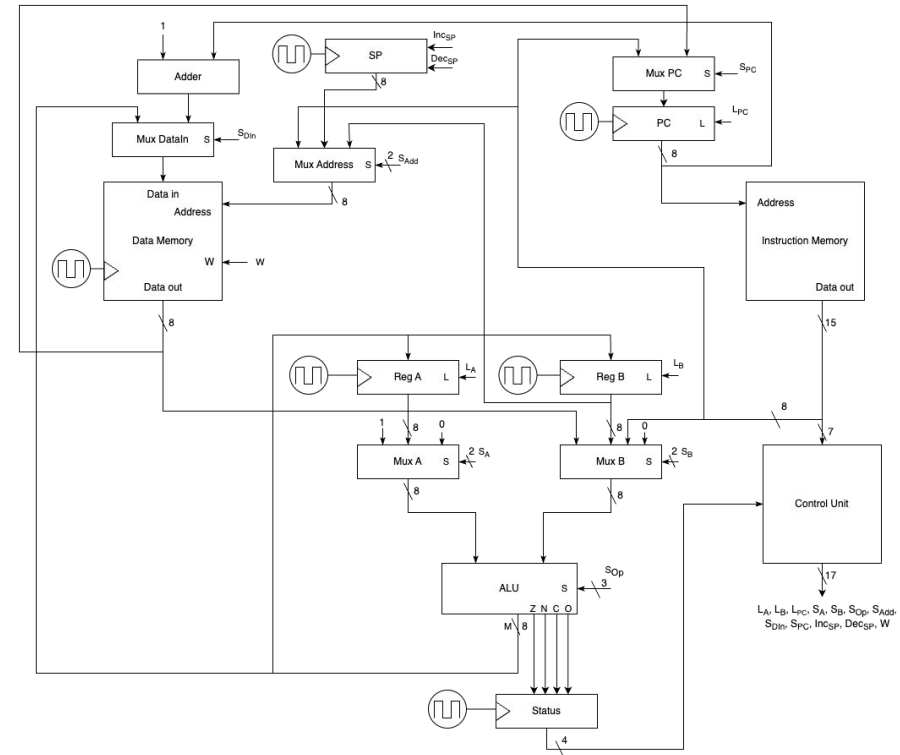
- **Clase 4 - Programabilidad**
- **Clase 5 - Saltos y Subrutinas**
- **Clase 7 - Arquitecturas de Computadores**
- **05 - Programabilidad (Apuntes)**
- **06 - Saltos y Subrutinas (Apuntes)**
- **07 - Arquitecturas de Computadores (Apuntes)**

When your friend only writes programs in assembly language



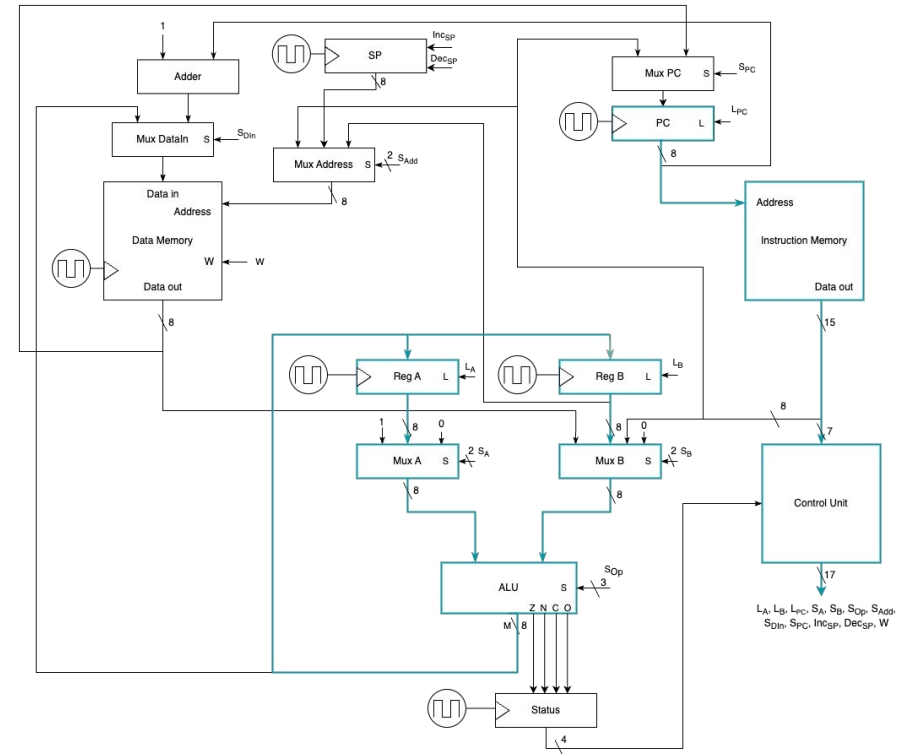
- **Diagrama del computador básico**

Diagrama completo. Iremos destacando las conexiones y componentes relevantes que habilitan cada funcionalidad.



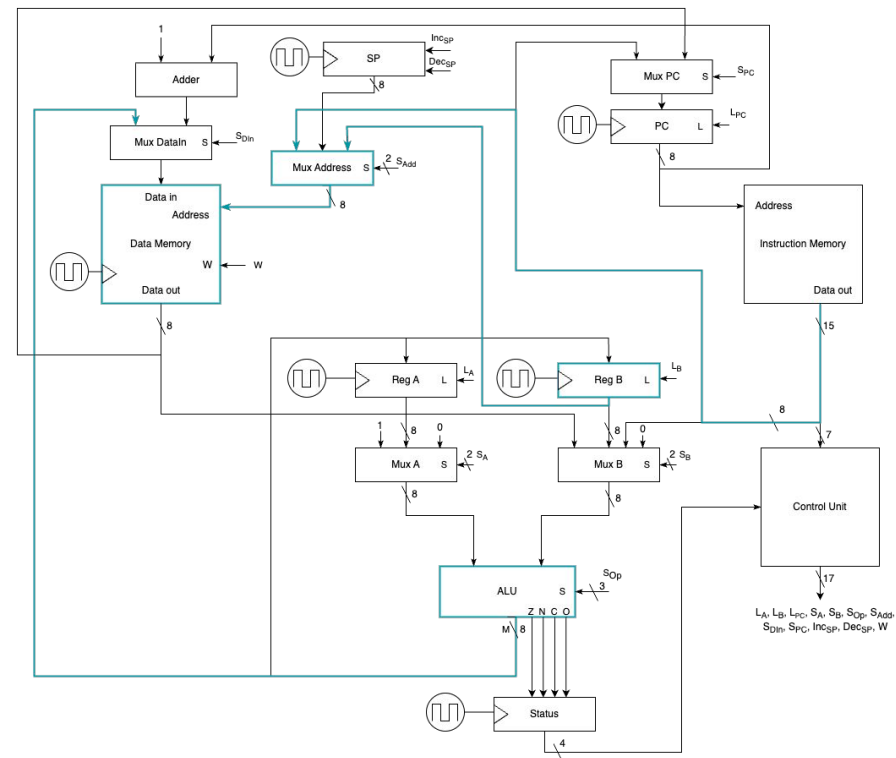
Computador básico - Microarquitectura

- Contador PC (*Program Counter*) permite ejecutar de forma secuencial un programa almacenado en la memoria ROM *Instruction Memory*. Por cada flanco de subida incrementa su valor y, así, cambia la instrucción en ejecución.
- *Control Unit* decodifica el *opcode* de la instrucción en señales de control que ejecutan la operación deseada.
- Registros *A* y *B* permiten realizar operaciones en la ALU y almacenar el resultado temporalmente. Muxes *A* y *B* permiten ampliar las operaciones a realizar.



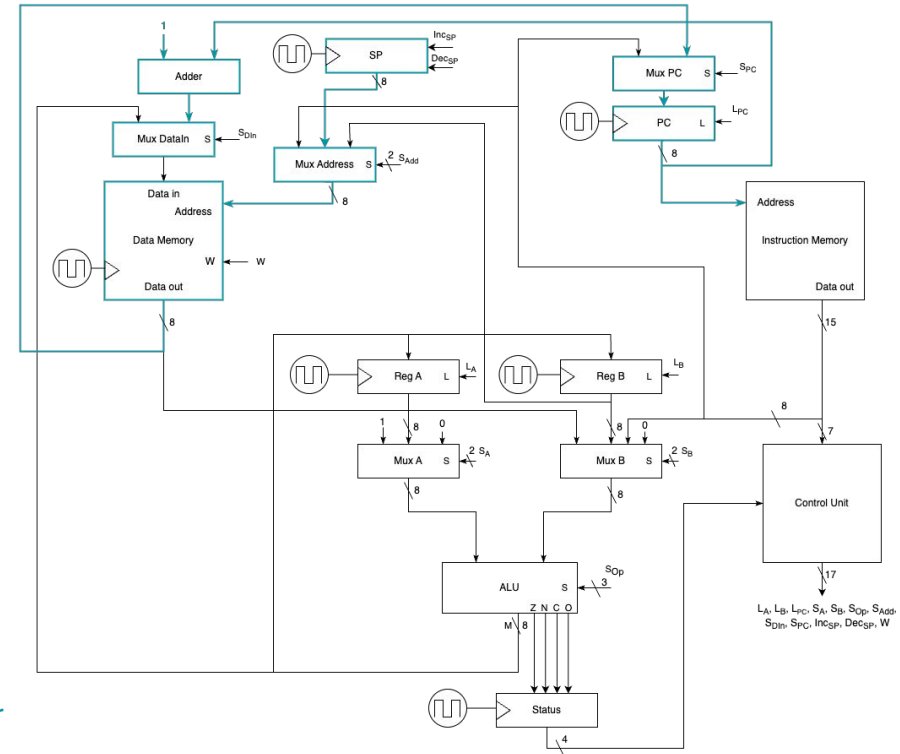
Computador básico - Microarquitectura

- Valor literal (numérico) asociado a la instrucción puede ser utilizado para realizar más operaciones a través del Mux B .
- Los resultados de las operaciones se pueden almacenar en la memoria RAM *Data Memory* (donde también se almacenan las variables del programa).
- La dirección donde se lee o escribe un dato puede ser dada por el literal mismo (direccionamiento directo) o por el valor del registro B (direccionamiento indirecto).



Computador básico - Microarquitectura

- Contador SP (*Stack Pointer*) permite añadir **memoria de stack** (ingreso de elementos desde la última dirección de la memoria).
- A través de la memoria de *stack* podemos implementar **subrutinas**. La conexión del PC y la *Data Memory* permite almacenar la dirección de retorno (PC+1) en el *stack* y cargarla de vuelta a través de la conexión entre la salida de la *Data Memory* y PC a través de su Mux.



* Por construcción, SP siempre apunta una dirección sobre el tope. Por ende, para leer el valor del tope es necesario invertir una iteración en decrementarlo en una unidad.

Computador básico - ISA

- **Instruction Set Architecture:** Indica la **forma** en la que se escriben los programas de una arquitectura y las operaciones que soporta, indicando a su vez las señales requeridas para ello, su formato, etc.

Instrucción	Operandos	Opcode	Condición	L _{PC}	L _A	L _B	S _{A0,1}	S _{B0,1}	S _{Op0,1,2}	S _{Add0,1}	S _{DIn}	S _{PC}	W	Inc _{SP}	Dec _{SP}
MOV	A,B	0000000		0	1	0	ZERO	B	ADD	-	-	-	0	0	0
	B,A	0000001		0	0	1	A	ZERO	ADD	-	-	-	0	0	0
	A,Lit	0000010		0	1	0	ZERO	LIT	ADD	-	-	-	0	0	0
	B,Lit	0000011		0	0	1	ZERO	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0000100		0	1	0	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	B,(Dir)	0000101		0	0	1	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	(Dir),A	0000110		0	0	0	A	ZERO	ADD	LIT	ALU	-	1	0	0
	(Dir),B	0000111		0	0	0	ZERO	B	ADD	LIT	ALU	-	1	0	0
	A,(B)	0001000		0	1	0	ZERO	DOUT	ADD	B	-	-	0	0	0
	B,(B)	0001001		0	0	1	ZERO	DOUT	ADD	B	-	-	0	0	0
	(B),A	0001010		0	1	0	A	ZERO	ADD	B	ALU	-	1	0	0

Parte 1 de la ISA del computador básico

Computador básico - ISA

ADD	A,B	0001011	0	1	0	A	B	ADD	-	-	-	0	0	0
	B,A	0001100	0	0	1	A	B	ADD	-	-	-	0	0	0
	A,Lit	0001101	0	1	0	A	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0001110	0	1	0	A	DOUT	ADD	LIT	-	-	0	0	0
	A,(B)	0001111	0	1	0	A	DOUT	ADD	B	-	-	0	0	0
	(Dir)	0010000	0	0	0	A	B	ADD	LIT	ALU	-	1	0	0
SUB	A,B	0010001	0	1	0	A	B	SUB	-	-	-	0	0	0
	B,A	0010010	0	0	1	A	B	SUB	-	-	-	0	0	0
	A,Lit	0010011	0	1	0	A	LIT	SUB	-	-	-	0	0	0
	A,(Dir)	0010100	0	1	0	A	DOUT	SUB	LIT	-	-	0	0	0
	A,(B)	0010101	0	1	0	A	DOUT	SUB	B	-	-	0	0	0
	(Dir)	0010110	0	0	0	A	B	SUB	LIT	ALU	-	1	0	0
AND	A,B	0010111	0	1	0	A	B	AND	-	-	-	0	0	0
	B,A	0011000	0	0	1	A	B	AND	-	-	-	0	0	0
	A,Lit	0011001	0	1	0	A	LIT	AND	-	-	-	0	0	0
	A,(Dir)	0011010	0	1	0	A	DOUT	AND	LIT	-	-	0	0	0
	A,(B)	0011011	0	1	0	A	DOUT	AND	B	-	-	0	0	0
	(Dir)	0011100	0	0	0	A	B	AND	LIT	ALU	-	1	0	0
OR	A,B	0011101	0	1	0	A	B	OR	-	-	-	0	0	0
	B,A	0011110	0	0	1	A	B	OR	-	-	-	0	0	0
	A,Lit	0011111	0	1	0	A	LIT	OR	-	-	-	0	0	0
	A,(Dir)	0100000	0	1	0	A	DOUT	OR	LIT	-	-	0	0	0
	A,(B)	0100001	0	1	0	A	DOUT	OR	B	-	-	0	0	0
	(Dir)	0100010	0	0	0	A	B	OR	LIT	ALU	-	1	0	0

Parte 2 de la ISA del computador básico

Computador básico - ISA

NOT	A,A	0100011	0	1	0	A	-	NOT	-	-	-	0	0	0
	B,A	0100100	0	0	1	A	-	NOT	-	-	-	0	0	0
	(Dir)	0100101	0	0	0	A	B	NOT	LIT	ALU	-	1	0	0
XOR	A,B	0100110	0	1	0	A	B	XOR	-	-	-	0	0	0
	B,A	0100111	0	0	1	A	B	XOR	-	-	-	0	0	0
	A,Lit	0101000	0	1	0	A	LIT	XOR	-	-	-	0	0	0
	A,(Dir)	0101001	0	1	0	A	DOUT	XOR	LIT	-	-	0	0	0
	A,(B)	0101010	0	1	0	A	DOUT	XOR	B	-	-	0	0	0
	(Dir)	0101011	0	0	0	A	B	XOR	LIT	ALU	-	1	0	0
SHL	A,A	0101100	0	1	0	A	-	SHL	-	-	-	0	0	0
	B,A	0101101	0	0	1	A	-	SHL	-	-	-	0	0	0
	(Dir)	0101110	0	0	0	A	B	SHL	LIT	ALU	-	1	0	0
SHR	A,A	0101111	0	1	0	A	-	SHR	-	-	-	0	0	0
	B,A	0110000	0	0	1	A	-	SHR	-	-	-	0	0	0
	(Dir)	0110001	0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
INC	B	0110010	0	0	1	ONE	B	ADD	-	-	-	0	0	0
	(B)	0110011	0	0	1	ONE	DOUT	ADD	B	ALU	-	1	0	0
	(Dir)	0110100	0	0	1	ONE	DOUT	ADD	LIT	ALU	-	1	0	0

Parte 3 de la ISA del computador básico

Computador básico - ISA

CMP	A,B	0110101		0	0	0	A	B	SUB	-	-	-	0	0	0
	A,Lit	0110110		0	0	0	A	LIT	SUB	-	-	-	0	0	0
	A,(Dir)	0110111		0	0	0	A	DOUT	SUB	LIT	-	-	0	0	0
	A,(B)	0111000		0	0	0	A	DOUT	SUB	B	-	-	0	0	0
JMP	Dir	0111001		1	0	0	-	-	-	-	-	LIT	0	0	0
JEQ	Dir	0111010	Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JNE	Dir	0111011	Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JGT	Dir	0111100	N=0 y Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLT	Dir	0111101	N=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JGE	Dir	0111110	N=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLE	Dir	0111111	N=1 o Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JCR	Dir	1000000	C=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JOV	Dir	1000001	V=1	1	0	0	-	-	-	-	-	LIT	0	0	0
CALL	Dir	1000010		1	0	0	-	-	-	SP	PC	LIT	1	0	1
RET		1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000100		1	0	0	-	-	-	SP	-	DOUT	0	0	0
		1000101		0	0	0	A	ZERO	ADD	SP	ALU	-	1	0	1
PUSH	B	1000110		0	0	0	ZERO	B	ADD	SP	ALU	-	1	0	1
POP	A	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000111		0	1	0	ZERO	DOUT	ADD	SP	-	-	0	0	0
POP	B	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
NOP		1001001		0	0	0	-	-	-	-	-	-	0	0	0

Parte 4 de la ISA del computador básico

Computador básico - ISA

- Ejemplo de código con la ISA del computador básico. Cabe destacar que se definen dos segmentos: DATA para las variables (incluyendo arreglos) y CODE para las instrucciones.
- El **Assembler** es el programa encargado de transformar este código en lenguaje de máquina (binario) y de asegurar que existan instrucciones que almacenen las variables en memoria, así como también *mapear* correctamente las direcciones de los *labels* a los literales correctos.

DATA:

```
var 1  
arr 1  
    3  
    5
```

CODE:

```
MOV B,arr  
MOV A,(B)  
SHL A,A  
MOV (B),A  
INC B  
MOV A,(B)  
SHL A,A  
MOV (B),A  
INC B  
MOV A,(B)  
SHL A,A  
MOV (B),A
```

Computador básico - Clasificaciones

■ Paradigmas según ISA

- **RISC:** *Reduced Instruction Set Computer*. Instrucciones pequeñas y simples. Su diseño permite simplificar el *hardware*, poniendo énfasis en el *software*.
- **CISC:** *Complex Instruction Set Computer*. Muchas instrucciones y con complejidad alta. Énfasis en un *hardware* más complejo para poder ejecutarlas.

■ Paradigmas según memoria

- **Harvard:** Memoria de datos separada de la memoria de instrucciones.
- **Von Neumann:** Datos e instrucciones en una única unidad de memoria.

* Bajo estos paradigmas, el computador básico presenta una microarquitectura Harvard y una ISA RISC.

Arquitectura RISC-V

- Contenido que encuentran en:
 - **Clase 8 - Arquitectura RISC-V**



Arquitectura RISC-V - Instrucciones

■ Instrucciones más utilizadas: Operaciones lógico-aritméticas

Mnemotecnia	Instrucción	Tipo	Descripción
ADD rd, rs1, rs2	Adición	R	$rd \leftarrow rs1 + rs2$
SUB rd, rs1, rs2	Sustracción	R	$rd \leftarrow rs1 - rs2$
ADDI rd, rs1, imm12	Adición de literal	I	$rd \leftarrow rs1 + imm12$
AND/OR/XOR rd, rs1, rs2	Operación AND/OR/XOR	R	$rd \leftarrow rs1 \& / / ^ rs2$
ANDI/ORI/XORI rd, rs1, imm12	Operación AND/OR/XOR con literal	I	$rd \leftarrow rs1 \& / / ^ imm12$
SLL/SRL rd, rs1, rs2	Operación <i>shift left/right</i> lógico	R	$rd \leftarrow rs1 \ll / \gg rs2$
SRA rd, rs1, rs2	Operación <i>shift right</i> aritmético	R	$rd \leftarrow rs1 \gg rs2$
SLLI/SRLI rd, rs1, shamt	Operación <i>shift left/right</i> lógico con literal	I	$rd \leftarrow rs1 \ll / \gg shamt$
SRAI rd, rs1, shamt	Operación <i>shift right</i> aritmético con literal	I	$rd \leftarrow rs1 \gg shamt$

* *shamt* o *shift amount* es la cantidad de *shifts* a realizar y se codifica como un entero a partir de los 5 bits menos significativos del literal ($imm12[4:0]$).

Arquitectura RISC-V - Instrucciones

- **Instrucciones más utilizadas:** Operaciones de carga, almacenamiento y subrutinas

Mnemotecnia	Instrucción	Tipo	Descripción
LW rd, imm12(rs1)	Cargar <i>word</i> (32 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
SW rs2, imm12(rs1)	Almacenar <i>word</i> (32 bits)	S	$rs2 \rightarrow \text{mem}[rs1 + \text{imm12}]$
BEQ rs1, rs2, imm12	Salto con condición “igual”	B	if $rs1 == rs2$: $PC \leftarrow PC + \text{imm12}$
BNE rs1, rs2, imm12	Salto con condición “distinto”	B	if $rs1 \neq rs2$: $PC \leftarrow PC + \text{imm12}$
BGE rs1, rs2, imm12	Salto con condición “mayor o igual”	B	if $rs1 \geq rs2$: $PC \leftarrow PC + \text{imm12}$
BLT rs1, rs2, imm12	Salto con condición “menor”	B	if $rs1 < rs2$: $PC \leftarrow PC + \text{imm12}$
JAL rd, imm20	Salto incondicional con “enlace”	J	$rd \leftarrow PC+4$; $PC \leftarrow PC + \text{imm20}$
JALR rd, imm12(rs1)	Salto incondicional con “enlace” a registro	I	$rd \leftarrow PC+4$; $PC \leftarrow rs1 + \text{imm12}$

* En general, en vez de JAL y JALR trataremos de usar pseudo-instrucciones al ser más intuitivas. LW también se puede usar para leer una dirección de memoria a partir de su *label*.

Arquitectura RISC-V - Instrucciones

■ Instrucciones más utilizadas: Pseudo-instrucciones

Mnemotecnia	Instrucción	Instruccion(es) base
LI rd, imm12	Cargar literal en registro que utiliza ≤ 12 bits	ADDI rd, zero, imm12
LA rd, sym	Cargar dirección en registro	AUIPC rd, sym[31:12]; ADDI rd, rd, sym[11:0]
MV rd, rs	Copiar registro	ADDI rd, rs, 0
NOT rd, rs	Complemento de 1	XORI rd, rs, -1
NEG rd, rs	Complemento de 2	SUB rd, zero, rs
BGT rs1, rs2, offset	Salto si $rs1 > rs2$	BLT rs2, rs1, offset
BLE rs1, rs2, offset	Salto si $rs1 \leq rs2$	BGE rs2, rs1, offset
BEQZ rs1, offset	Salto si $rs1 = 0$	BEQ rs1, zero, offset
BNEZ rs1, offset	Salto si $rs1 \neq 0$	BNE rs1, zero, offset
J offset	Salto incondicional	JAL zero, offset
CALL offset12	Llamado a subrutina (dirección ≤ 12 bits)	JALR ra, ra, offset12
RET	Retorno de la subrutina	JALR zero, 0(ra)

Arquitectura RISC-V - Directivas

■ Directivas básicas para correr código en *Assembly* RISC-V

Directiva de Assembler	Descripción
<code>.text</code>	Segmento de texto (código).
<code>.data</code>	Sección de datos global.
<code>.globl sym</code>	<i>Label</i> sym se vuelve global.
<code>.word w1, w2, ..., wN</code>	Almacena <i>N</i> valores de 32 bits en palabras de memoria sucesivas.
<code>.byte w1, w2, ..., wN</code>	Almacena <i>N</i> valores de 8 bits en bytes de memoria sucesivos.

■ ECALLs para imprimir enteros y terminar programa

```
.globl main
.text
main:
    li a0, 11           # a0 = 11
    li a7, 1            # a7 = 1 (PrintInt)
    ecall               # Imprime 11 en consola
    li a7, 10           # a7 = 10 (Exit)
    ecall               # Termina el programa
```

Arquitectura RISC-V - Convención

■ Registros relevantes y encargado de respaldo en subrutinas

Registro(s)	Mnemotecnia ABI	Descripción	Encargado de respaldo
x0	zero	Registro cero. Almacena este valor y no cambia . Ignora las escrituras.	-
x1	ra	Return Address . Almacena la dirección de retorno de las subrutinas.	<i>Caller</i>
x2	sp	Stack Pointer , apunta al último elemento almacenado.	<i>Callee</i>
x5-x7, x28-x31	t0-t6	Registros temporales. Pierden su valor entre llamados de subrutinas.	<i>Caller</i>
x8-x9, x18-x27	s0-s11	Registros guardados (<i>saved</i>). Preservan su valor entre llamados de subrutinas.	<i>Callee</i>
x10-x17	a0-a7	Registros para argumentos de subrutinas.	<i>Caller</i>
x10-x11	a0-a1	Si bien son de argumentos de subrutinas, también se utilizan para almacenar valores de retorno.	<i>Caller</i>

Arquitectura RISC-V - Convención

■ Ejemplo de respaldo de registros t*

En este caso, los registros t0-t1 son respaldados por el *caller* porque son modificados por la subrutina y por definición son *caller-saved*.

* En estricto rigor es necesario respaldar el registro ra solo para llamadas de subrutinas anidadas, pero por convención lo respaldamos siempre ante un `call` dado que nuestro código *podría* llamarse desde otro archivo, por lo que respaldar ra asegura que este llamado siempre funcione.

* Este código almacena un segundo arreglo cuyos elementos poseen el valor duplicado de los elementos de arr.

```
.data
len: .word 5
arr: .word 198, 137, 42, 63, 175
.text
start:
    li s0, 4                # s0 = 4 bytes por dirección
    lw s1, len              # s1 = largo del arreglo
    li t0, 0                # Contador (i)
    la t1, arr              # t1 = dirección del arreglo (inicialmente arr[0])
    mul t2, s0, s1          # t2 = s0 * s1 = bytes que ocupa el arreglo de entrada
    add t2, t2, t1          # t2 += t1 = primera dirección que podemos usar de .data
    while:
        lw a0, 0(t1)        # a0 = arr[i]
        addi sp, sp, -12    # Respaldamos t0, t1 y ra (caller-saved). t1 se respalda
        sw t0, 0(sp)        # aunque no se use porque call lo modifica con la
        sw t1, 4(sp)        # dirección de retorno en RARS
        sw ra, 8(sp)
        call double_give_next_person
        lw t0, 0(sp)        # Recuperamos t0, t1 y ra y restauramos el stack
        lw t1, 4(sp)
        lw ra, 8(sp)
        addi sp, sp, 12
        sw a0, 0(t2)        # out[i] = a0
        addi t0, t0, 1      # t0 += 1
        beq t0, s1, end     # Termina cuando se recorre todo el arreglo
        add t1, t1, s0      # t1 += s0 = dirección arr[i+1]
        add t2, t2, s0      # t2 += s0 = dirección out[i+1]
        j while
    double_give_next_person:
        mv t0, a0
        add a0, a0, t0      # a0 = a0 + t0 = 2 * a0
        ret
end:
    li a7, 10
    ecall
```

Arquitectura RISC-V - Convención

■ Ejemplo de respaldo de registros s*

Si en el mismo código invertimos el uso de registros s* por registros t*, entonces los debemos respaldar en la subrutina ya que son *callee-saved* (lo que asegura que su valor persista entre llamados).

* Es el mismo código anterior. Estratégicamente no usamos t1 porque call modifica su valor en RARS y necesitaría respaldo.

```
.data
len: .word 5
arr: .word 198, 137, 42, 63, 175
.text
start:
    li t0, 4                # s0 = 4 bytes por dirección
    lw t2, len              # s1 = largo del arreglo
    li s0, 0                # Contador (i)
    la s1, arr              # t1 = dirección del arreglo (inicialmente arr[0])
    mul s2, t0, t2          # t2 = s0 * s1 = bytes que ocupa el arreglo de entrada
    add s2, s2, s1          # t2 += t1 = primera dirección que podemos usar de .data
    while:
        lw a0, 0(s1)        # a0 = arr[i]
        addi sp, sp, -4     # Respalamos ra (callee-saved)
        sw ra, 0(sp)
        call double_give_next_person
        lw ra, 0(sp)        # Recuperamos ra y restauramos el stack
        addi sp, sp, 4
        sw a0, 0(s2)        # out[i] = a0
        addi s0, s0, 1      # s0 += 1
        beq s0, t2, end     # Termina cuando se recorre todo el arreglo
        add s1, s1, t0      # s1 += t0 = dirección arr[i+1]
        add s2, s2, t0      # s2 += t0 = dirección out[i+1]
        j while
double_give_next_person:
    addi sp, sp, -4         # Respalamos s0 (callee-saved)
    sw s0, 0(sp)
    mv s0, a0              # a0 = a0 + s0 = 2 * a0
    add a0, a0, s0          # Recuperamos s0 y restauramos el stack
    lw s0, 0(sp)
    addi sp, sp, 4
    ret
end:
    li a7, 10
    ecall
```

Arquitectura RISC-V - Convención

■ Ejemplo de respaldo de función recursiva

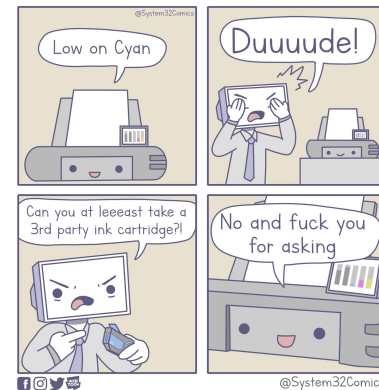
En este ejemplo, es importante notar que la función actúa como *caller* y como *callee*.

```
.data
N: .word 4                # N = Argumento de factorial. Calcularemos N! = 4
.text
main:
    addi sp, sp, -4        # Reservamos 4 bytes en el stack
    sw ra, 0(sp)           # Respalamos ra
    la t0, N               # Dirección de memoria de N
    lw t0, 0(t0)           # Valor de N
    add a0, zero, t0       # Argumento 0 = valor de N
    call factorial         # factorial(N)
    li a7, 1               # Llamada de sistema: print int
    ecall                  # Valor en consola: 24 (a0, valor de retorno)
    lw ra, 0(sp)           # Restauramos ra
    addi sp, sp, 4         # Restauramos el stack
    li a7, 10              # Llamada de sistema: exit
    ecall

factorial:
    addi sp, sp, -8        # Reservamos 8 bytes en el stack
    sw ra, 0(sp)           # Respalamos ra
    sw a0, 4(sp)           # Respalamos N
    blez a0, factorial_zero # if (N > 0){
    addi a0, a0, -1        #     N -= 1
    call factorial         #     (N-1)! = factorial(N-1)
    lw t0, 4(sp)           #     Recuperamos N
    mul a0, a0, t0         #     N! = N * (N-1)! = N * factorial(N-1)
    j factorial_end       # }
factorial_zero:
    li a0, 1               # else {
    # N! = 1
    # }
    lw ra, 0(sp)           # Restauramos solo ra, a0 ahora posee el retorno
    addi sp, sp, 8         # Restauramos el stack
    ret
```

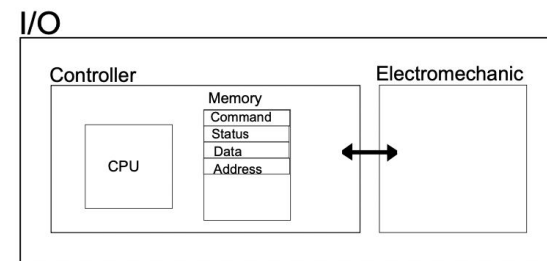
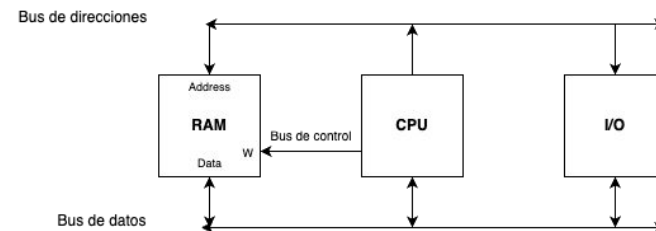
Dispositivos I/O

- Contenido que encuentran en:
 - **Clase 9 - Comunicación de CPU y Memoria con I/O**
 - **09 - Comunicación de CPU y Memoria con I/O (Apuntes)**



Dispositivos I/O - Diagrama general

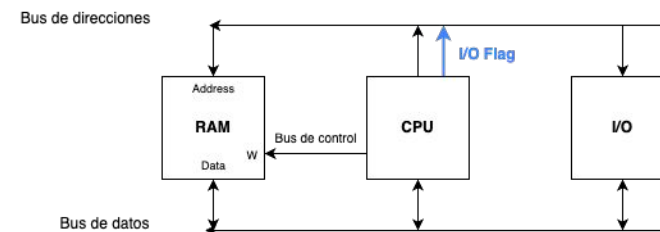
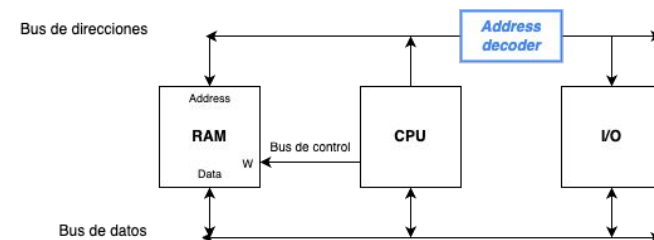
- **Conectividad:** Los dispositivos I/O comparten el bus de datos de la memoria y la CPU y se puede interactuar con ellos con el bus de direcciones.
- **Composición:** Son como un computador, pero con tareas específicas para interactuar con sus piezas electromecánicas. Los registros más importantes son los de **estado** y **comandos** para interactuar con ellos.



* Con el registro de estado podemos obtener información sobre el dispositivo, mientras que con el registro de comando podemos interactuar con él para que realice ciertas tareas. Los registros de datos y direcciones se utilizan para dispositivos de almacenamiento, de forma que puedan transferir o recibir datos de la memoria del computador.

Dispositivos I/O - Comunicación con CPU

- **Memory mapped:** Se reservan N direcciones de memoria que apuntan a registros de comando/estado de dispositivos I/O. Estos son detectados por el **address decoder**.
Problema: **memory barrier** (se pierde el uso de las direcciones reservadas en la RAM).
- **Port mapped:** Se agregan instrucciones a la ISA para leer (IN Reg, Port) o escribir (OUT Port, Reg) los valores de los registros de comando/estado de los dispositivos I/O a partir de puertos asociados a estos. Se añade una **flag** adicional para poder interpretar bien el puerto como un registro de un dispositivo.



Dispositivos I/O - Comunicación con CPU

■ Ejemplo de interacción con I/O

A continuación, se mostrará un ejemplo de código en RISC-V que interactúa con dispositivos *memory mapped* utilizados por un tren con las siguientes tabla de direcciones, estados, comandos y dirección base 0x10030000:

Offset	Nombre	Descripción
0x00	dir_ise	Contiene la dirección ISR del manejo de alerta del tren
0x04	reg_tr	Registro de comandos del tren
0x08	sensor_te	Registro de estado de sensor de temperatura
0x0C	sensor_pr	Registro de estado de sensor de protuberancia
0x18	sensor_dr	Registro de estado de sensor de derrame
0x20	rad_cl	Registro de comandos de la radio

Nombre	Comando o Estado	Valor
reg_tr	Notificar locomotora	255
reg_tr	Freno de emergencia	127
sensor_te	Temperatura alta	255
sensor_te	Temperatura baja	1
sensor_te	Temperatura normal	127
sensor_pr	Sobredimensión del tren o protuberancias	4
sensor_pr	Rieles libres de obstáculos	2
sensor_dr	Existencia de derrame	3
sensor_dr	Sin derrame de fluidos en el tren	1
rad_cl	Mandar reporte de sensores	1

Dispositivos I/O - Comunicación con CPU

En esta ISR, se notifica a la locomotora de una emergencia y se activa el freno del tren si se detecta una protuberancia en los rieles usando las direcciones de los registros señalados en la diapositiva anterior. Lo importante es notar cómo se revisan los estados y se realizan comandos leyendo/escribiendo datos en las direcciones mapeadas de los dispositivos.

```
isr:
    addi sp, sp, -12           # Registros s* son callee-saved. También respaldamos ra por llamada anidada
    sw s0, 0(sp)
    sw s1, 4(sp)
    sw ra, 8(sp)
    call sens_pr_sub
    mv s0, a0                 # s0 = estado sensor_pr
    li s1, 0                  # s1 = cantidad de estados no deseados
    li t0, 4                  # Revisión de protuberancias
    beq s0, t0, emergency_action # Protocolo de emergencia si hay protuberancias
    j isr_end
emergency_action:
    call rad_cl_sub
    call reg_tr_sub
    j isr_end
sens_pr_sub:
    li t0, 0x1003000c         # t0 = dirección sensor_pr
    lw a0, 0(t0)              # a0 = valor sensor_pr
    ret
rad_cl_sub:
    li t0, 0x10030020         # t0 = dirección rad_cl
    li t1, 1                  # t1 = comando para enviar reporte de sensores
    sw t1, 0(t0)
    ret
reg_tr_sub:
    li t0, 0x10030004         # t0 = dirección reg_tr
    li t1, 255                # t1 = comando para notificar a la locomotora
    sw t1, 0(t0)              # t1 = comando para activar el freno de emergencia
    li t1, 127
    sw t1, 0(t0)
    ret
isr_end:
    lw s0, 0(sp)              # Registros s* y ra se restauran
    lw s1, 4(sp)
    lw ra, 8(sp)
    addi sp, sp, -12          # Se restablece el stack
    mret                      # Retorno de una excepción para RISC-V (usado para ISRs)
```

Dispositivos I/O - Modos de comunicación

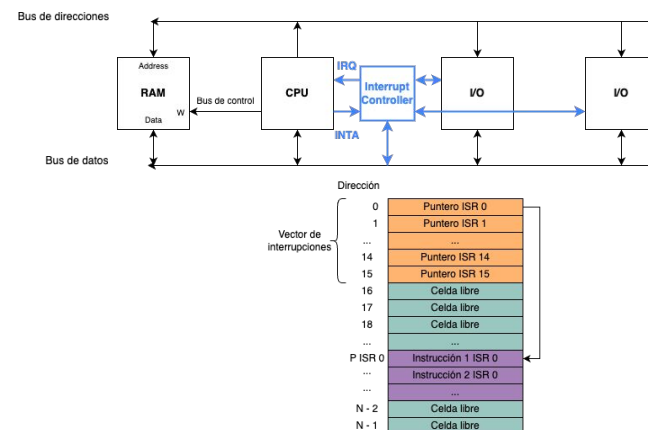
- **Polling:** La CPU consulta con cierta frecuencia el estado de un dispositivo para saber si debe realizar una acción.

Ej: Programa en RISC-V que espera a que una impresora prenda para imprimir.

```
.text
main:
    li t0, 112          # t0 = 0x70 = Registro de estado de una impresora
    lw t1, 0(t0)         # t0 = Estado de una impresora
    li s0, 1            # Estado 1 = Prendida.
    li s1, 3            # Estado 3 = Prendiendo.
    beq t1, s0, print    # Está prendida, imprimimos.
    beq t1, s1, waitTurnOn # Está prendiendo, esperamos a que prenda.
    j end
waitTurnOn:
    lw t1, 0(t0)
    beq t1, s0, print    # Si todavía no está prendida, repetimos la pregunta.
    j waitTurnOn
print:
    ...
```

- **Interrupciones:** Los dispositivos I/O solicitan una “interrupción” que gatilla una **Interruptation Subroutine** o **ISR** para realizar una acción. Esta solicitud la envían como una señal **Interruptation Request** o **IRQ** al **controlador de interrupciones**, que se encarga de solicitar la atención de la CPU. Si es atendida, la CPU envía la señal **Interruptation Acknowledge** o **INTA** y el controlador le envía de vuelta el **identificador del dispositivo** para acceder a la dirección de su ISR desde el **vector de interrupciones**.

La CPU requiere de una **Interruptation Flag** o **IF** para saber cuándo está atendiendo una interrupción (**IF = 1** → CPU disponible para atender interrupciones).



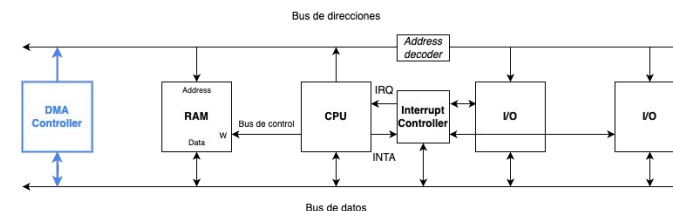
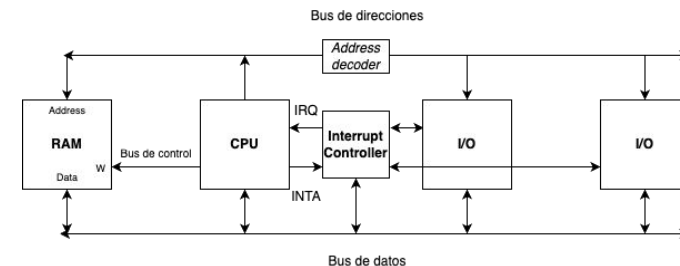
Dispositivos I/O - Modos de comunicación

■ Tipos de interrupción

- **Excepciones:** Condiciones de error al ejecutar una instrucción atendidas por el sistema operativo a través de ISRs específicas: *exception handlers*. **Ejemplos:** división por cero, *stack overflow*.
- **Traps:** Llamadas explícitas al sistema operativo que gatillan interrupciones que le ceden el control a este para que realice una acción. **Ejemplo en RISC-V:** `ecall`

Dispositivos I/O - Transferencia de datos

- **Programmed I/O (PIO):** La CPU se encarga de gestionar la transferencia de datos a través de subrutinas que solo tienen ese fin. Cuenta con la desventaja de que los dispositivos I/O solo necesitan acceder a la memoria de datos (RAM), no requieren de procesamiento en la CPU.
- **Direct Memory Access (DMA):** Controlador con acceso al bus de datos que se encarga de gestionar la transferencia de datos entre la RAM y los dispositivos I/O. Solo notifica a la CPU del inicio y del término de la transferencia, permitiendo que esta ejecute otras tareas.



* El controlador DMA solo puede interactuar con dispositivos I/O memory mapped.

Caché

■ Contenido que encuentran en:

- **Clase 10 - Jerarquía de Memoria y Memoria Caché**
- **10 - Jerarquía de Memoria y Memoria Caché (Apuntes)**

My CPU when the L1 cache misses



Caché - Definiciones

- **Jerarquía de memoria:** En vez de conectarse con una única unidad de memoria, la CPU se comunica con una jerarquía de unidades conectadas entre sí que van aumentando en tamaño, pero disminuyendo en velocidad de transferencia.
- **Principios de localidad:** Se opta por partir con una unidad de memoria pequeña pero rápida (**memoria caché**) ya que al programar se cumplen **dos principios de localidad**:
 - **Espacial:** Si accedo a una dirección de memoria, probablemente acceda a datos **cercanos** a ella.
 - **Temporal:** Si accedo a una dirección de memoria, probablemente acceda a ella de nuevo **en el corto plazo**.

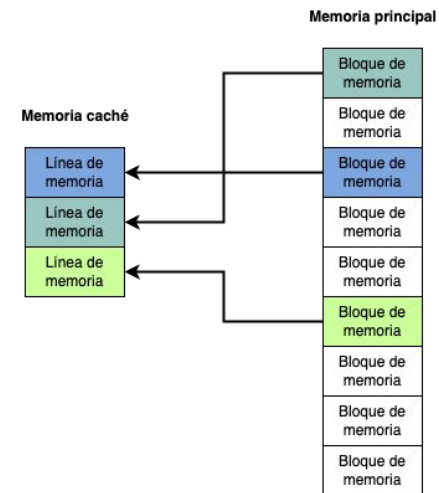


```
.data
arr: .word 1, 0, 6, 12, 1
len: .word 5
index: .word 0
avg: .word 0
.text
main:
    lw t0, index
    lw t1, len
    lw t2, avg
    la t3, arr
    while:
        beq t0, t1, end
        lw t4, 0(t3)
        add t2, t2, t4
        addi t0, t0, 1
        addi t3, t3, 4
        j while
    end:
        div t2, t2, t1
        la t5, avg
        sw t2, (t5)
```

* En este ejemplo de código, se evidencia en naranjo el principio de localidad espacial porque al acceder al valor de un arreglo, es probable (y acertado) que acceda a las direcciones contiguas, correspondientes a sus otros elementos. En morado se evidencia el principio de localidad temporal porque al acceder al índice y largo del arreglo, es probable (y acertado) que acceda a sus direcciones de nuevo para usar sus datos de nuevo (en este caso, para finalizar la iteración).

Caché - Definiciones

- **Caché:** Corresponde a la memoria más cercana a la CPU en la jerarquía. Se divide físicamente en **líneas** que almacenan **bloques de la memoria principal**, que son una división lógica de la RAM. Ambos deben poseer el mismo tamaño.
- **Controlador de caché:** Componente que se encarga de realizar las operaciones de búsqueda de datos en la caché y la transferencia de bloques de la memoria principal a las líneas en caso de no encontrarse el dato. Esto permite que la CPU solicite datos de memoria sin tener conocimiento de la existencia de la jerarquía. Posee dos funciones principales: **mecanismo de acceso a datos** (mapeo de bloques a líneas) y **política de escritura** (actualización de datos de la caché y la memoria principal).



* La transferencia de bloques de memoria contiguos a la caché aprovecha el principio de localidad espacial. Adicionalmente, cada línea de la caché cuenta con un bit de validez que indica si su contenido es válido o no para ser utilizado. Por defecto todas las líneas parten con este bit en cero.

Caché - Controlador de caché

El mecanismo de acceso de datos del controlador se define a través de dos criterios:

- **Función de correspondencia:** Criterio de asociación (o mapeo) de un bloque de memoria a una línea de la caché.
- **Política de reemplazo:** Criterio de línea a reemplazar para sobrescribir un bloque de memoria en caso de tener la caché llena. En este caso se busca aprovechar el **principio de localidad temporal**.

Caché - Funciones de correspondencia

Fórmula utilizada para asociar un bloque a una caché. Depende de:

- Tamaño de línea de caché (en palabras de memoria - bytes).
- Cantidad de líneas en la caché.
- Tamaño de la memoria principal (cantidad de direcciones).

Salvo que se indique lo contrario, **se asumirá que una palabra de memoria es de 1 byte**. Veremos las tres funciones estudiadas usando de ejemplo una memoria principal de 16 bytes (2^4 direcciones, 4 bits de dirección) y una caché de 8 bytes con líneas de 2 bytes (2^1 palabras por línea, 1 bit de *offset*).

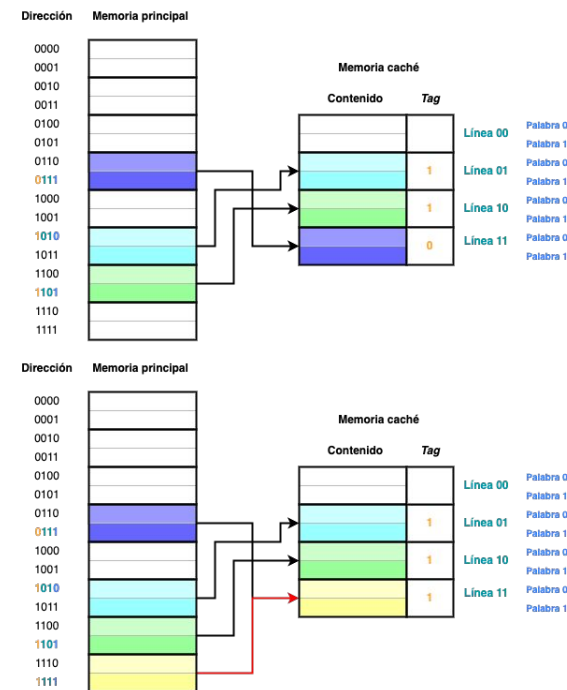
Caché - Funciones de correspondencia

Directly Mapped: Un bloque de memoria se puede asociar únicamente a una línea de la caché. Los bits de una dirección se separan de la siguiente forma: **tag**|**id línea**|**offset**

Si tenemos 4 líneas, usamos $\log_2(4) = 2$ bits para el índice de línea.

Ej: Dirección 7 = **0111** → Su bloque se almacena en la línea 11.

* El **tag** corresponde al identificador del bloque dentro de una línea. Por construcción, para todas las funciones es el que nos permite identificar de forma certera si un bloque se encuentra contenido en la caché o no.



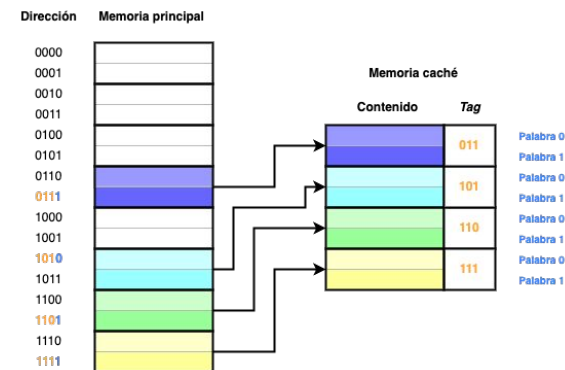
* En esta función no se utilizan políticas de reemplazo, si otro bloque necesita copiarse a la caché y se mapea en una línea ocupada, se sobrescribe (como se muestra en la imagen).

Caché - Funciones de correspondencia

Fully associative: Un bloque de memoria se puede asociar a cualquier línea de la caché. Los bits de una dirección se separan de la siguiente forma: **tag**|**offset**

Como tenemos un bit de *offset*, el resto será utilizado para el *tag*.

Ej: Dirección 7 = **0111** → Su *tag* es 011 y puede encontrarse en cualquier línea.



* La desventaja de esta función es que se deben recorrer todas las líneas de la caché para poder determinar si el bloque al que pertenece una dirección de memoria se encuentra disponible o no.

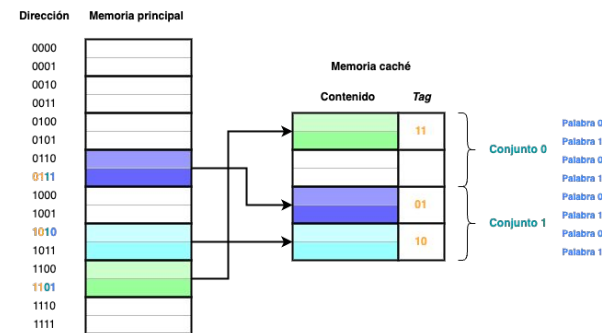
Caché - Funciones de correspondencia

N-way associative: La caché se agrupa en conjuntos de N líneas y un bloque de memoria se puede asociar a cualquier línea dentro del conjunto al que pertenece. Los bits de una dirección se separan de la siguiente forma:

tag|**id** **conjunto**|**offset**

Para *2-way associative*, contamos con $4 \div 2$ conjuntos, por lo que su id usa $\log_2(2) = 1$ bit.

Ej: Dirección 7 = **0111** → Su *tag* es 01 y puede encontrarse en cualquier línea del conjunto 1.



* Esta es la más usada en la práctica por ser la más equilibrada entre las ventajas de *directly mapped* y *fully associative*.

En esta función, siempre se tiene que:

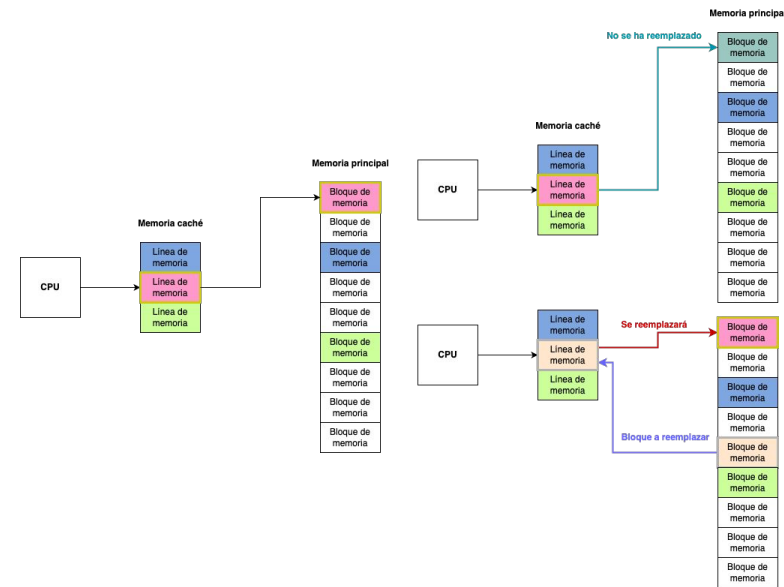
$$\# \text{ Conjuntos} = \# \text{ Líneas} \div N$$

Caché - Políticas de reemplazo

- **Bélády:** Se saca el bloque que se utilizará más lejos en el futuro. **Ideal no alcanzable en la práctica.**
- ***First-In First-Out (FIFO)*:** El primer bloque en entrar es el primero en salir.
- ***Least Frequently Used (LFU)*:** El bloque con menos accesos se saca.
- ***Least Recently Used (LRU)*:** El bloque con mayor tiempo sin accesos se saca. **¡El más usado en la práctica!**
- ***Random*:** Rápido, con rendimiento inferior a LFU/LRU pero mejor que FIFO.

Caché - Políticas de escritura

- **Write-through:** Si se modifica un bloque de memoria contenido en la caché, se actualiza inmediatamente en la memoria principal.
- **Write-back:** Si se modifica un bloque de memoria contenido en la caché, se actualiza en la memoria principal solo al momento de ser reemplazado.



Ejemplo de write-through vs. write-back.

Caché - Tipos de memoria

- **Caché unified:** Mezcla en sus líneas bloques de datos con bloques de instrucciones.
- **Caché split:** Separa la mitad de sus líneas para almacenar bloques de datos y utiliza el resto para bloques de instrucciones.

Esta división interesa en computadores **Von Neumann** que poseen una única unidad de memoria.

Caché - Evaluación de una memoria caché

Las memorias caché se evalúan a partir del tiempo de acceso promedio, que se rige a partir de la siguiente fórmula:

Tiempo promedio en caso de *hit*

Tiempo promedio en caso de *miss* *

$$\begin{aligned} TP &= HR * HT + (1 - HR) * (HT + MP) \\ &= HT + (1 - HR) * MP \end{aligned}$$

- TP = Tiempo promedio de acceso
- HR = *Hit rate* promedio (# *Hits* ÷ # Accesos)
- HT = *Hit time*, tiempo promedio de acceso en la caché
- MP = *Miss penalty*, tiempo adicional para buscar el bloque de memoria en caso de *miss*.

Multiprogramación

- Contenido que encuentran en:
 - **Clase 11 - Multiprogramación**
 - **11 - Multiprogramación (Apuntes)**

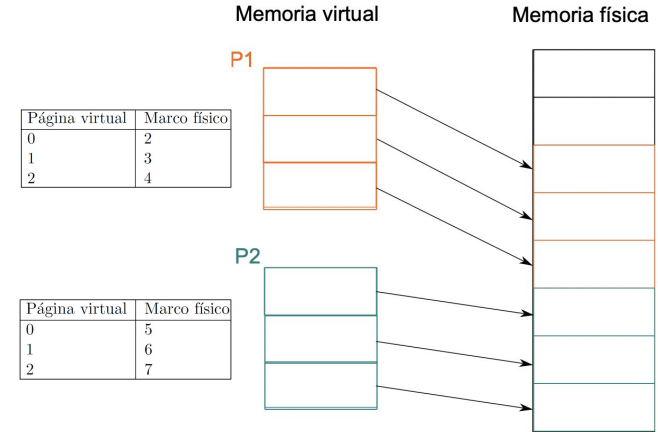


?



Multiprogramación - Memoria virtual

- Los procesos (programas en ejecución) hacen uso de **direcciones virtuales**, un espacio direccionable que posteriormente se traduce a **direcciones físicas** (direcciones reales de la memoria principal). Esto permite la ejecución de múltiples procesos sin necesidad de asignarles espacios de memoria limitados.
- **Memory Management Unit (MMU)**: Unidad encargada de hacer las traducciones de direcciones virtuales a las direcciones físicas asignadas a cada proceso a través de **esquemas de paginación**, donde cada uno de estos posee una **tabla de páginas** que almacena las asociaciones.

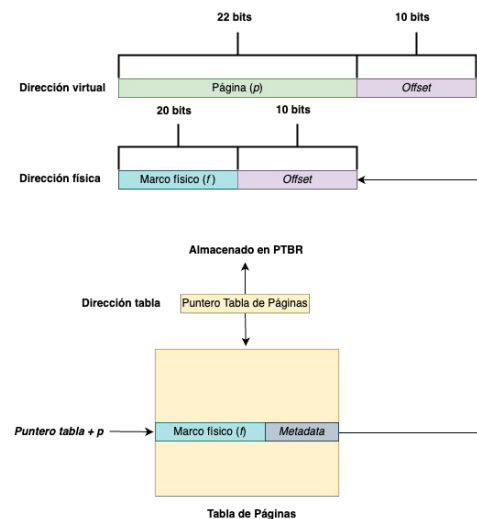


* En un esquema de paginación, una página virtual (bloque contiguo del espacio direccionable virtual) se asocia a un marco físico (bloque contiguo del espacio direccionable físico). Esta división por bloques es similar a la utilizada en la caché y permite almacenar una cantidad de información significativamente menor para llevar a cabo las traducciones (se traducen bloques completos con una sola entrada en vez de una única dirección).

Multiprogramación - Traducción de dirección virtual a física

En un esquema de direccionamiento virtual, siempre conoceremos el tamaño del espacio, del que se deduce la cantidad de bits de una dirección virtual. Luego, usamos el **tamaño de página** para obtener los **bits de offset** $-\log_2(\text{Tamaño página en bytes})-$. Separamos los bits de la dirección virtual así: **número de página**|**offset**

Utilizamos el número de página como índice de la tabla de páginas del proceso que solicita la dirección para acceder a la entrada que poseerá los **bits de marco físico**. Finalmente, se reemplaza el número de página por estos bits para obtener la dirección física: **número de marco**|**offset**



* **Ejemplo de traducción para direcciones virtuales de 32 bits, direcciones físicas de 30 bits y tamaño de página de 1KB. En este caso, se requiere de un registro especial: Page Table Base Register (PTBR). Este almacena la dirección de la tabla de páginas del proceso que esté corriendo en la CPU.**

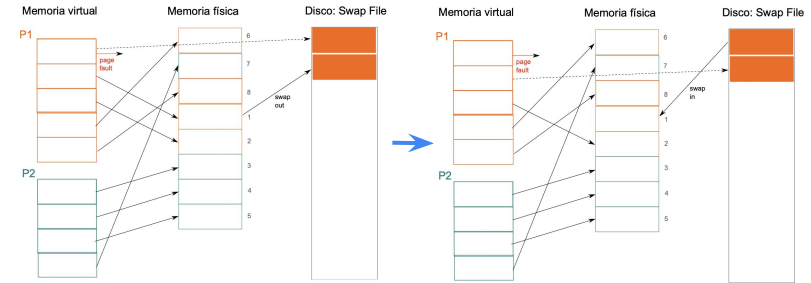
Multiprogramación - Bits de *metadata*

Cada entrada de la tabla de páginas poseen bits de *metadata* que entregan información relevante respecto a la traducción:

- ***Present bit***: Indica, para una entrada válida, si el contenido de la página está en un marco físico o si está en el ***swap file*** (disco).
- ***Dirty bit***: Indica si el contenido de la página se ha modificado o no. Si no es el caso, en caso de ***swap out*** no se copia el contenido al ***swap file***.

Multiprogramación - *Page fault*

Al momento de buscar el marco físico para una página, incurrimos en un *page fault* si la página posee un marco físico ubicado en el *swap file* (*Present bit* = 0).



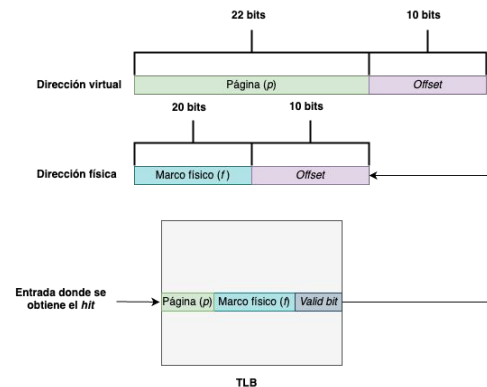
Página virtual	Marco físico	Disco
0	3	0
1	3	1
2	4	0
3	0	0
4	2	0
5	x	0
6	x	0
7	x	0

* El marco físico a reemplazar se guarda en el *swap file* (*swap out*) y luego se trae de vuelta el marco solicitado (*swap in*). Se utilizan las mismas políticas de reemplazo estudiadas en la caché.

Multiprogramación - *Translation Lookaside Buffer*

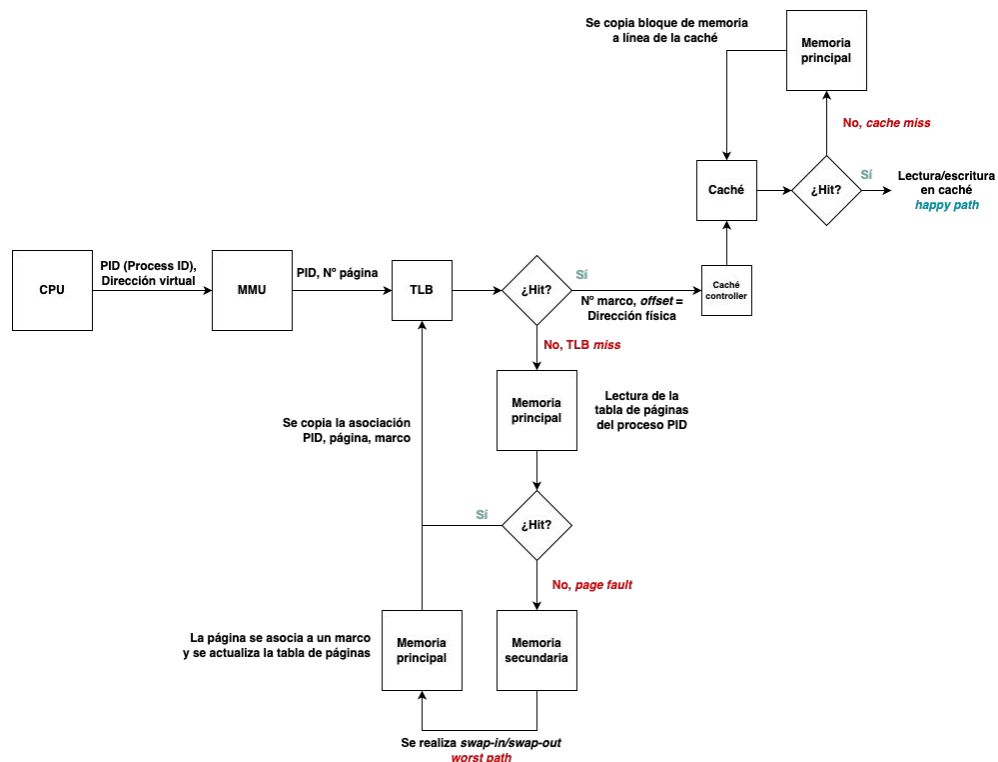
Al usar memoria virtual, se requieren dos accesos a memoria (uno para acceder a la tabla de páginas y otro para obtener el dato). Para reducir los tiempos de acceso, se usa una caché *fully associative* llamada **Translation Lookaside Buffer (TLB)**. Esta almacena para cada una de sus entradas:

- Bits del número de página como **tag**.
- Bits del número de marco como **contenido**.
- Bit de validez.



* Ejemplo de traducción para direcciones virtuales de 32 bits, direcciones físicas de 30 bits y tamaño de página de 1KB haciendo uso de una TLB.

Multiprogramación - Flujo de acceso de memoria completo



Multiprogramación - Tamaño de tablas de página

Como cada proceso posee su propia tabla de páginas, es importante conocer la forma de calcular su tamaño y considerar esquemas de paginación alternativos para reducirlo. Este se calcula con las siguientes fórmulas:

Para efectos prácticos:

- 1 byte = 1B
- $2^{10}\text{B} = 1\text{KB}$
- $2^{20}\text{B} = 2^{10}\text{KB} = 1\text{MB}$
- $2^{30}\text{B} = 2^{10}\text{MB} = 1\text{GB}$
- $2^{40}\text{B} = 2^{10}\text{GB} = 1\text{TB}$

Tamaño tabla de páginas = $\#PTE * \text{sizeof}(PTE)$

- $\#PTE = \# \text{ Páginas} = 2^{\#Bits \text{ número de página}}$
- $\text{sizeof}(PTE) = \#Bits \text{ número de marco} + \#Bits \text{ metadata}$

Ejemplo:

- Direcciones virtuales de 32 bits
- Direcciones físicas de 30 bits
- Tamaño de página 1KB ($\#Bits \text{ offset} = \log_2(2^{10}) = 10$)
- $\#Bits \text{ metadata} = 4$

$\#Bits \text{ número de página} = \#Bits \text{ dirección virtual} - \#Bits \text{ offset} = 22$

$\#Bits \text{ número de marco} = \#Bits \text{ dirección física} - \#Bits \text{ offset} = 20$

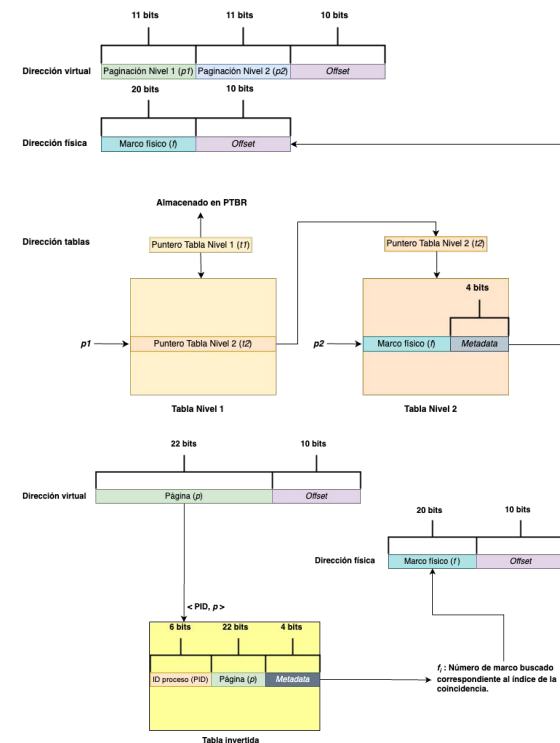
$\text{sizeof}(PTE) = 20 + 4 = 24\text{b} = 3\text{B}$

$\#PTE = 2^{22}$

Tamaño tabla de páginas = $2^{22} * 3\text{B} = 2^2 * 3 * 2^{20}\text{B} = 12\text{MB}$

Multiprogramación - Esquemas de paginación alternativos

- **Paginación multinivel:** Los bits de número de página se dividen por **niveles**. Cada segmento de bits corresponde al índice de la tabla de un nivel. Solo la tabla del último nivel posee la traducción de la dirección virtual, el resto posee punteros físicos a la tabla del nivel siguiente. Este esquema **reduce significativamente el tamaño usado de tablas de páginas por proceso**.
- **Tabla de páginas invertida:** Se utiliza una única tabla para todos los procesos. El índice de cada entrada corresponde al número de marco físico de la traducción (por ende, tiene tantas entradas como marcos físicos existan).



ILP

- Contenido que encuentran en:
 - **Clase 12 - Paralelismo a Nivel de Instrucción (ILP)**
 - **12 - Paralelismo a Nivel de Instrucción (ILP) (Apuntes)**

CPU: *predicts wrong execution branch*



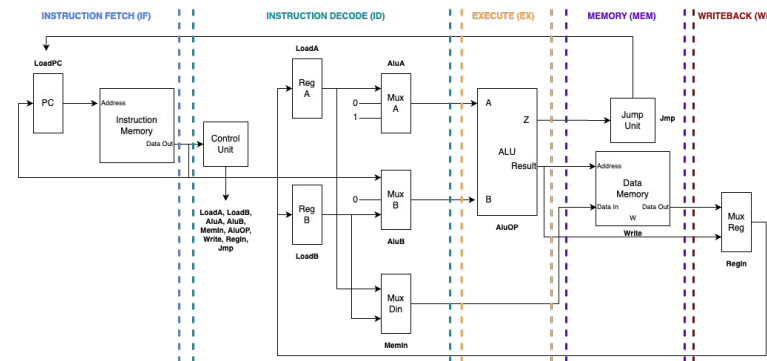
* Perdón por repetir el meme, pero estaba muy bueno.



ILP - Pipeline

Se busca ejecutar instrucciones **simultáneamente** en el computador básico. Para ello, primero se simplifica la arquitectura de forma que pueda separarse en **cinco etapas independientes**:

- **Instruction Fetch (IF)**: Obtención del *opcode* y el literal de la memoria de instrucciones.
- **Instruction Decode (ID)**: Decodificación del *opcode* en señales de control y selección de *inputs* de la ALU.
- **Execute (EX)**: Ejecución de la ALU.
- **Memory (MEM)**: Lectura o escritura de la memoria de datos.
- **Writeback (WB)**: Actualización de registros.



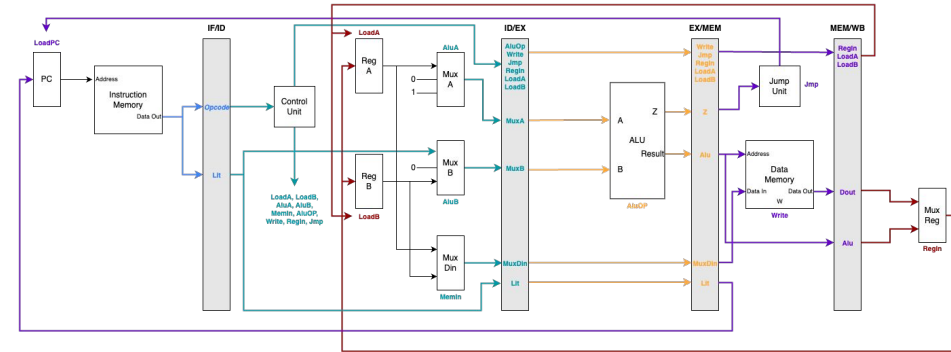
* Con esta simplificación, perdemos:

- Direccionamiento indirecto por registro B.
- Memoria de *stack*.
- Soporte de subrutinas.
- Saltos condicionales: JGT, JLT, JGE, JLE, JOV, JCR.
- Manejo de saltos por la unidad de control (ahora los maneja una unidad de saltos).
- Almacenamiento directo de resultados de la ALU en la memoria de datos (pasa de ser una arquitectura *Register-Memory* a *Register-Register* o *Load-Store*).

ILP - Pipeline

Para poder ejecutar cada etapa de forma separada, se agregan **registros intermedios** que almacenan las señales de control y los resultados de la etapa anterior para usarse en la siguiente etapa. Estos son: IF/ID, ID/EX, EX/MEM, MEM/WB.

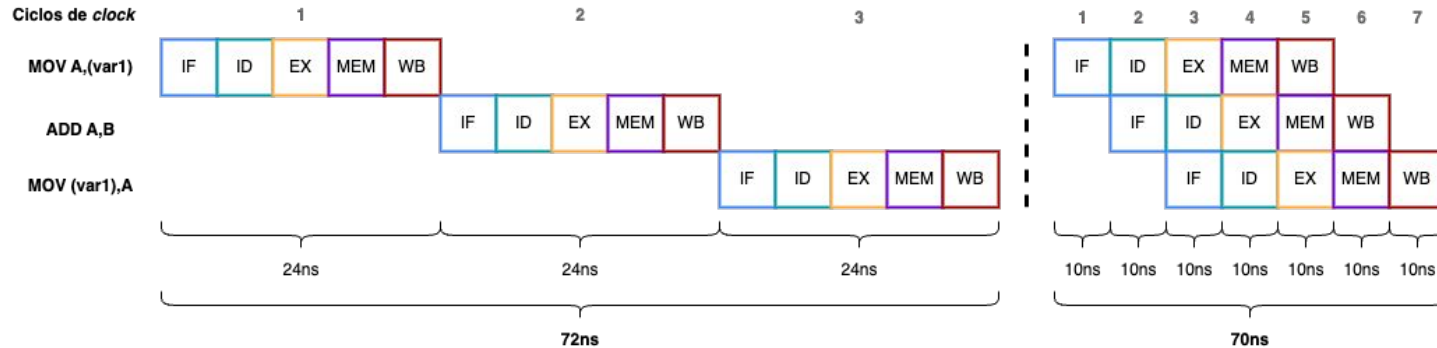
Por esta estructura, naturalmente ahora cada instrucción toma 5 ciclos en ejecutarse, pero cada ciclo toma menos tiempo de ejecución respecto a los del computador básico.



* Las observaciones más importantes:

- Las señales LoadA y LoadB que actualizan los registros provienen de la etapa WB y no de la unidad de control, estos se almacenan en el registro intermedio para propagarse y usarse al final.
- La señal LoadPC utilizada para saltos proviene de la *Jump Unit* en la etapa MEM.
- Al igual que cualquier otro registro, los registros intermedios se actualizan por flanco de subida.

ILP - Ejecución de instrucciones en el *Pipeline*



Como se observa en la imagen, cuando la primera instrucción pasa de IF a ID, la segunda puede empezar su ejecución en IF. Con 3 instrucciones se observa una disminución de tiempo, en programas reales la disminución es **muy significativa**.

ILP - Hazards

Corresponden a problemas que surgen de la dependencia natural entre instrucciones. Existen tres tipos:

- **Hazards de datos:** Ocurren cuando existen dependencias de datos entre instrucciones.
- **Hazards de control:** Ocurren con el control de flujo, *i.e.* con las instrucciones de salto.
- **Hazards estructurales:** Ocurren cuando dos etapas necesitan una misma unidad al mismo tiempo (**Ej:** IF y MEM en Von Neumann). **No existen en el computador básico con *pipeline*.**

ILP - Hazards de datos

Caso 1: Dependencia de datos entre dos instrucciones contiguas.

Resolución: *Forwarding* del resultado desde el registro EX/MEM a EX.

Detección: $EX/MEM.LoadX == 1$ and $ID/EX.AluX == X$, $X = A/B$

Caso 2: Dependencia de datos entre una instrucción y la previa a la anterior.

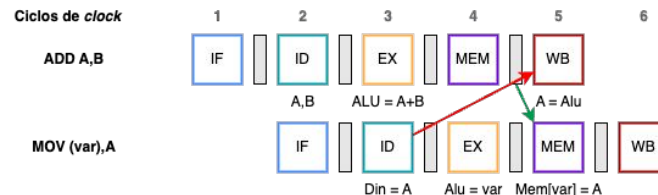
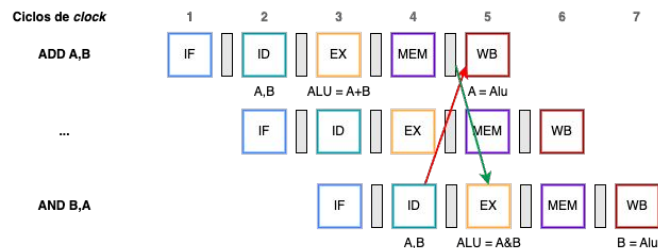
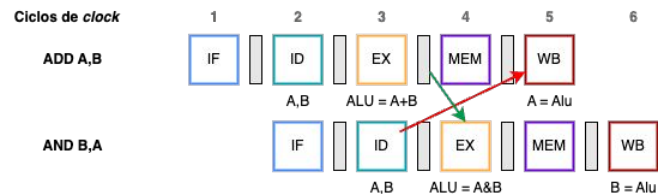
Resolución: *Forwarding* del resultado desde el registro MEM/WB a EX.

Detección: $MEM/WB.LoadX == 1$ and $ID/EX.AluX == X$ and
 $EX/MEM.LoadX == 0$, $X = A/B$

Caso 3: Dependencia de datos entre instrucciones contiguas con escritura de memoria.

Resolución: *Forwarding* del resultado desde el multiplexor Mux Reg a la etapa MEM.

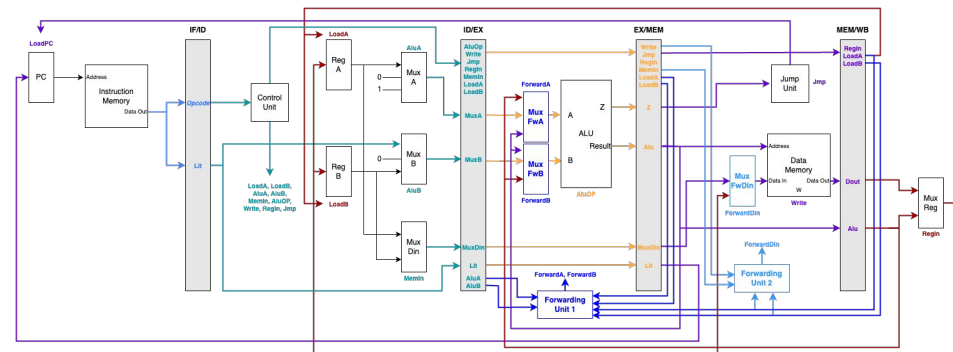
Detección: $MEM/WB.LoadX == 1$ and $EX/MEM.MemIn == X$
and $EX/MEM.Write == 1$, $X = A/B$



ILP - Hazards de datos

Para resolver los casos anteriores, se añaden las **Forwarding Units**, encargadas de detectar los *hazards* antes señalados y transmitir el resultado correcto si corresponde.

Asimismo, se añaden los multiplexores FwA, FwB y FwDin para poder seleccionar el valor que corresponda si es que existe o no un *hazard*.



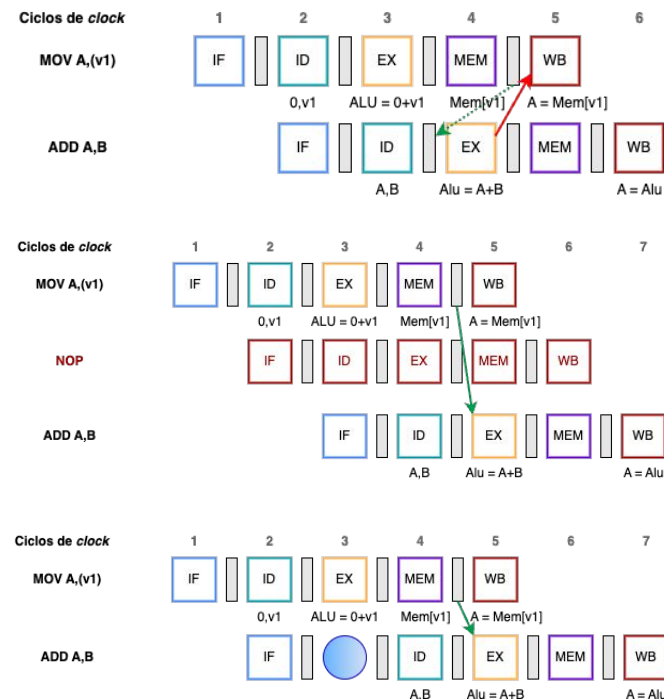
ILP - Hazards de datos

Caso 4: Dependencia de datos entre instrucciones contiguas con lectura de memoria.

Resolución: *Stalling* de un ciclo vía *software* (NOP) o *hardware* (inserción de burbuja) para resolver posteriormente con *forwarding*.

Detección: $ID/EX.LoadX == 1$ and $ID/EX.RegIn == Dout$
and $ID.AluX == X$, $X = A/B$

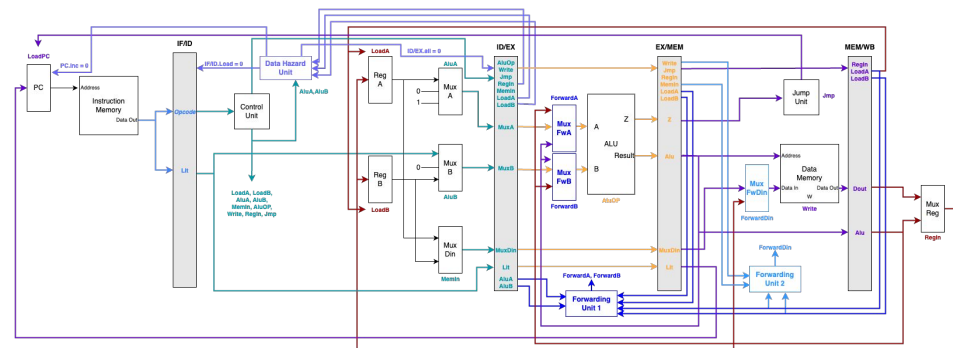
Este caso requiere la espera de un ciclo ya que el dato se obtiene al final de la etapa MEM, instante en que la instrucción anterior ya utilizó la unidad de ejecución.



ILP - Hazards de datos

Para resolver el caso anterior, se añade la **Data Hazard Unit**, encargada de insertar una burbuja y evitar por un ciclo el incremento del *Program Counter* para así alcanzar a resolver el *hazard*.

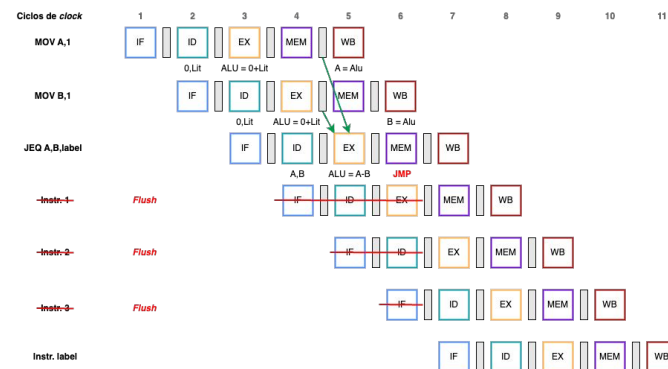
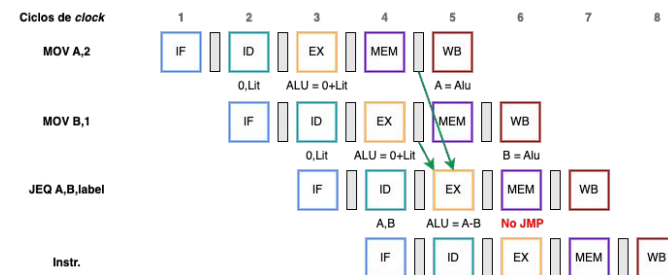
Si bien es importante conocer la unidad, en general estos casos los resolveremos por *software*.



ILP - Hazards de control

Ocurren con las instrucciones de salto. Estos se resuelven en MEM, *i.e.* después de tres ciclos de ejecución. Por eso, es necesario tomar una decisión respecto a qué instrucción ejecutar después. Se opta por utilizar una **unidad predictora de saltos** que decide si realizar el salto de inmediato o no. En caso de equivocarse (lo que se detecta en MEM), se debe hacer **flushing**, *i.e.* desechar el avance de las tres instrucciones ejecutadas erróneamente para comenzar con la ejecución de la instrucción correcta.

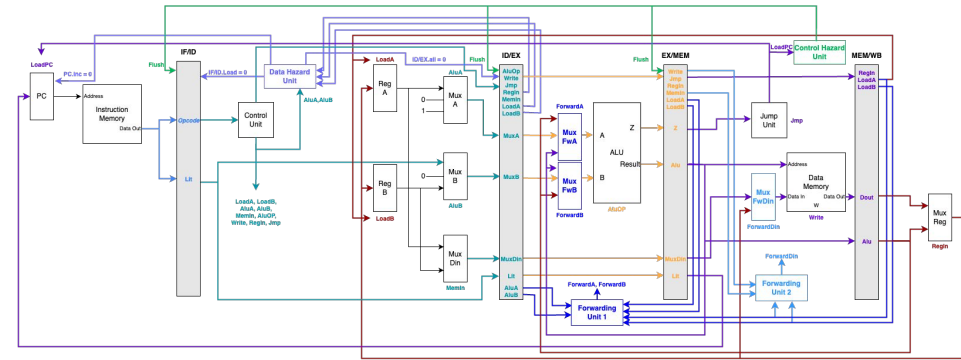
En el computador básico con *pipeline*, por simplicidad usaremos una unidad predictora de saltos que **siempre predice que este no ocurre**, en otro caso sería más complejo implementar en *hardware*.



ILP - Hazards de control

Para realizar *flushing*, se añade la **Control Hazard Unit**, encargada de realizar *reset* de los registros intermedios para evitar la ejecución efectiva de las instrucciones erróneas.

Esta corresponde a la versión completa del computador básico con *pipeline*.



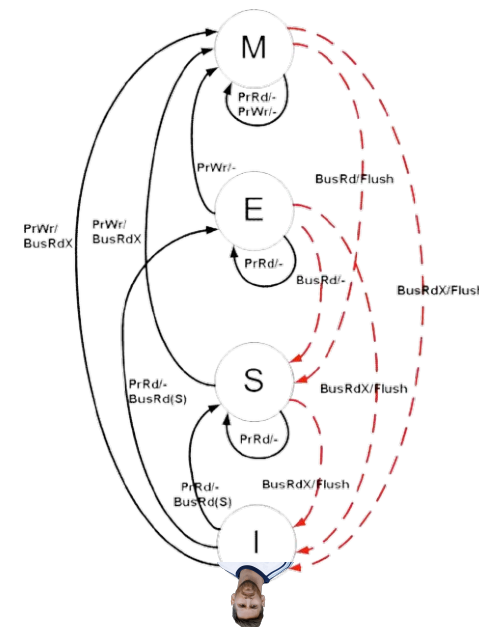
ILP - ISA del computador básico con *pipeline*

Instrucción	Operandos	Operación	Condiciones	Ejemplo de uso
MOV	A,B	A=B		-
	B,A	B=A		-
	A,Lit	A=Lit		MOV A,15
	B,Lit	B=Lit		MOV B,15
	A,(Dir)	A=Mem[Dir]		MOV A,(var1)
	B,(Dir)	B=Mem[Dir]		MOV B,(var2)
	(Dir),A	Mem[Dir]=A		MOV (var1),A
	(Dir),B	Mem[Dir]=B		MOV (var2),B
	A,(B)	A=Mem[B]		-
	B,(B)	B=Mem[B]		-
	(B),A	Mem[B]=A		-
ADD	A,B	A=A+B		-
	B,A	B=A+B		-
SUB	A,Lit	A=A-Lit		ADD A,5
	A,B	A=A-B		-
	B,A	B=A-B		-
AND	A,Lit	A=A-Lit		SUB A, 2
	A,B	A=A and B		-
	B,A	B=A and B		-
OR	A,Lit	A=A and Lit		AND A,15
	A,B	A=A or B		-
	B,A	B=A or B		-
NOT	A,Lit	A=A or Lit		OR A,5
	A,A	A=notA		-
XOR	B,A	B=notA		-
	A,A	A=A xor B		-
	B,A	B=A xor B		-
SHL	A,Lit	A=A xor Lit		XOR A,15
	A,A	A=shift left A		-
SHR	B,A	B=shift left A		-
	A,A	A=shift right A		-
INC	B,A	B=shift right A		-
	B	B=B+1		-
JMP	Dir	PC = Dir		JMP end
JEQ	Dir	PC = Dir	Z=1	JEQ label
JNE	Dir	PC = Dir	Z=0	JNE label
NOP				

* Si bien se asume que JEQ y JNE se realizan con la comparación entre los registros *A* y *B*, en los ejemplos de este material se explicitan para distinguir que ya no se realizan con el literal ni con la salida de memoria.

Coherencia de Caché

- Contenido que encuentran en:
 - **Clase 13 - Coherencia de Caché**
 - **13 - Paralelismo Avanzado (Apuntes)**



Coherencia de Caché

Si en una arquitectura de memoria compartida se utilizan cachés con protocolo de escritura **write-back**, entonces se presenta el problema de **coherencia de caché**: Si dos procesadores poseen en su caché un mismo bloque de datos, este debe ser consistente entre ellas para evitar falta de sincronización de datos al momentos de copiar de vuelta la línea en la memoria principal.

Para resolverlo, se implementan **protocolos** que se aseguran la coherencia. Estos requieren de **bus snooping**, que consiste en un bus compartido entre los controladores de caché para poder saber qué bloques de memoria se están leyendo o escribiendo desde otros procesadores.

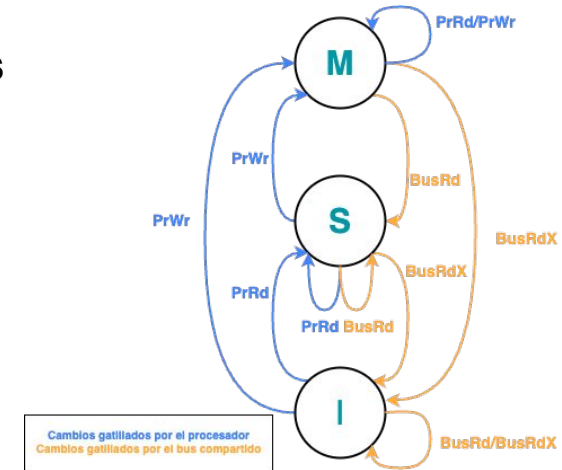
Coherencia de Caché - Protocolo MSI

Cada línea de la caché posee tres estados posibles:

- **Modified:** El contenido de la línea fue modificado y **no es consistente** con el bloque de la memoria principal. Este puede existir **solo en una caché** para un mismo bloque compartido.
- **Shared:** Indica que el contenido de la línea se encuentra **en al menos una memoria caché**, pero no se ha modificado.
- **Invalid:** Indica que el contenido de la línea es inválido, que puede ser por dato no existente o por solicitud desde el bus compartido.

Los estados cambian a partir de dos tipos de solicitud:

- **Solicitudes del procesador:** PrRd, PrWr
- **Solicitudes del bus compartido:** BusRd, BusRdX, Flush



* **Diagrama de estado de una línea de caché para el protocolo MSI.**

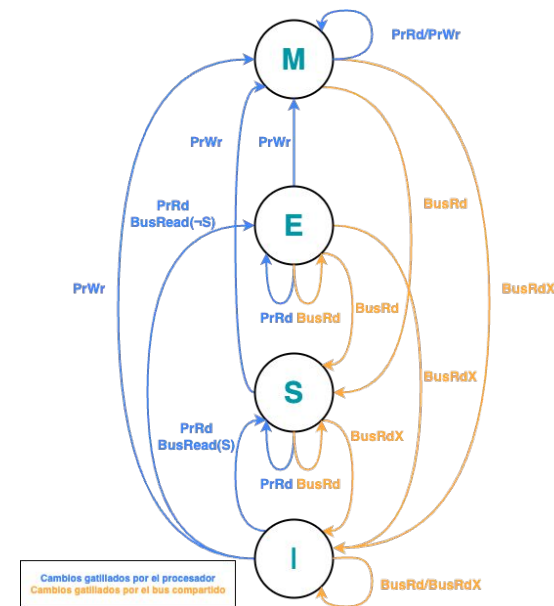
* **Importante:** Una línea en estado M siempre se transfiere a la memoria al cambiar a estado S o I (Flush).

Coherencia de Caché - Protocolo MESI

Agrega un nuevo estado al protocolo MSI y cambia el funcionamiento del estado **Shared**:

- **Shared**: Indica que el contenido de la línea se encuentra en **más de una caché** (al menos dos).
- **Exclusive**: Indica que el contenido de la línea se encuentra **exclusivamente en dicha caché**, pero sin modificaciones.

Es más eficiente respecto al tráfico del bus compartido ya que no gatilla la señal BusRdX para invalidar líneas en caso de escritura de una línea en estado **Exclusive**.



* Diagrama de estado de una línea de caché para el protocolo MESI.

FIN

Mi familia y yo les damos las gracias por su asistencia a clases y su interés en el curso. ❤️🥰



Créditos animalísticos



Minecraft
(Randall Biermann)



Rocky
(Cristóbal Iturriaga)



Mango
(Santiago Ernst)



Freddy
(Raimundo Murúa)



Ambar y Ágata
(Antonia Soto)



Pugwie y Arigato
(Germán Contreras)



Lisa, Inquilino y V
(Esteban Contreras)



Odín y Pisco
(Juan Ignacio García)



Milo, Milo chiquito y Tony
(Tomás López)



Shiro, Gatita Chica y Felipito
(Vicente Cuitiño)



Koda
(Nicolás Herrera)



Sokka
(Javiera Pinto)



Scooby y Baco
(Camila Argomedo)



Vicky, Pepita, Rayitas, O'Malley y Luchín
(Gabriel Venegas)



Benito y Chimú
(Giacomo Pasquali)



Cromis, Luna, Petra y Ramiro
(Joaquín Peralta)



Sparky
(José Mendoza)



Atila
(Doménica Hoppe)



Mocka y Lupe
(Daniela Ríos)



Maíalda (Mafy)
(Nicolás Poli)



Felipito Camiroaga y Frijolito
(Martina Mangiamarchi)





IIC2343

Arquitectura de Computadores

Clase 14 - Resumen del Curso

Profesor: Germán Leandro Contreras Sagredo